A Program Logic for Contextual Refinement of Imperative Shift/Reset Programs

ANONYMOUS AUTHOR(S)

The delimited control operators shift and reset enable convenient, direct-style manipulation of control flow, but their non-local effects interact in complex ways with higher-order functions and mutable state, complicating reasoning. Existing verification approaches require auxiliary specifications, such as protocols and invariants, which are problem-specific and nontrivial to provide. In this paper, we propose an algebraic approach based on contextual refinement. This enables simple reasoning in the presence of complex features, such as answer-type modification and multishot continuations, using standard proof techniques. To extend the approach to imperative shift/reset programs, we develop a relational program logic using a novel biorthogonal logical relation. Another novelty of our framework is that it is based on Indirection Theory, a more lightweight step-indexed foundation. To demonstrate the soundness and practical utility of the approach, we mechanise the metatheory of the system in Lean and use it to build an interactive verification tool. We evaluate it on a suite of case studies from the literature. The results show that the verifier can be used to establish contextual refinement for imperative shift/reset programs in a simple and effective manner, without auxiliary specifications.

CCS Concepts: • **Theory of computation** → **Logic and verification**; **Program specifications**.

Additional Key Words and Phrases: Separation Logic, Delimited Continuations, Deductive Verification, Automated Verification

1 Introduction

The control operators shift and reset were introduced by Danvy and Filinski [1990] as a means of programming with delimited continuations in *direct-style*, eliminating the need to work explicitly in continuation-passing style (CPS). They enable control-related language features — such as exceptions, nondeterminism, and coroutines — to be implemented simply as libraries, and have since found their way into many languages, including Racket [Flatt et al. 2007], Scala [Rompf et al. 2009], and OCaml [Kiselyov 2010].

The following example [Materzok and Biernacki 2011] illustrates their use.

$$\text{"Alice"} \mathbin{\hat{}} \langle \text{"has "} \mathbin{\hat{}} \mathit{shift}\ k.\ (k\ \text{"a dog"}) \mathbin{\hat{}} \text{" and the dog"} \mathbin{\hat{}} (k\ \text{"a bone."}) \rangle$$

Intuitively, a *shift* $k.\ e$ expression captures its continuation up to the nearest enclosing reset (denoted by $\langle\cdot\rangle$) and binds it to k in e, which may then manipulate it. Concretely, the continuation here may be thought of as the function $(\lambda s.\ \text{"has "} \mathbin{\hat{}} s)$, and this particular e invokes it twice. The result is "*Alice has a dog and the dog has a bone*", a neat demonstration of linguistic continuations.

As continuations captured by shift are first-class, it is possible to use one any number of times and in arbitrary ways, including not at all, as the next example demonstrates. Fig. 1 shows two programs which compute the product of a list of integers. `times` is conventional but inefficient: since 0 is absorbing for multiplication, a 0 anywhere in the list means that the result must be 0, yet `times` always performs a linear number of multiplications, potentially many of them by 0. In contrast, `times2` uses a shift to *abandon* its continuation as soon as a 0 is encountered, seemingly "returning" immediately to the enclosing reset — analogously to an exception.

There are two main families of approaches to reasoning about shift and reset. A type system was proposed early on by Danvy and Filinski [1989], its key idea being to track the *answer types* of an expression. It remains the basis of modern type-and-effect systems for temporal properties [Sekiyama and Unno 2023] and functional correctness [Kawamata et al. 2024]. Approaches in this family are typically highly automated but only work on pure programs, without heap manipulation.

The other line of work is in program logics based on separation logic, which capably handles heap manipulation. The challenge here is in dealing with non-locality. The pioneering work in this space, HTT_{cc} [Delbianco and Nanevski 2013], lacked a general frame rule for this reason, relying instead on *large-footprint* heap assertions. Timany and Birkedal [2019] articulated the problem as a lack of *context-locality*, owing to the inadmissibility of the standard bind rule; their *non-context-local triples* permit switching to global reasoning when continuations are involved. Both works also focus on the simpler, undelimited callcc/throw operators.

The state-of-the-art approach came from de Vilhena and Pottier [2021], who overcame the context-locality issue, in the closely-related setting of effect handlers, using the idea of a *protocol*, an abstraction of the context of an effect, and the related idea of a *handler invariant* [Soares and Pereira 2023]. de Vilhena [2022] proposed an encoding of shift and reset into effects and handlers, thereby extending the use of their logic to the former; the key idea, in shift/reset parlance, is that of an "answer postcondition": a specification for an enclosing reset. This idea of protocols (aka *theories*, as they ascribe meaning to otherwise uninterpreted effects) remains the dominant approach to specifying and reasoning about effect handlers [de Vilhena et al. 2026].

```
let rec times xs =
  match xs with
  | [] -> 1
  | x :: ys -> x * times ys

let rec times_sh xs =
  match xs with
  | [] -> 1
  | x :: ys ->
      if x = 0 then shift k 0
      else x * times_sh ys

let times2 xs =
  reset (times_sh xs)
```

Fig. 1. Computing the product of a list of numbers

Protocols enable modular verification, but are not in themselves modular *specifications*, as they bake in knowledge about how the handler operates. Furthermore, they can be quite non-trivial to write, involving the construction of intricate resource algebras to thread state through, particularly in the case of multishot continuations [de Vilhena 2022]. Another line of work proposed *staged logic* [Foo et al. 2024; Song et al. 2024], a logic for reasoning about the combination of effect handlers and imperative programs. It is notable for advancing an approach not based on protocols, instead using refinement directly to specify effectful behaviour. However, it has not been mechanised.

In this paper, we adapt and clarify the staged logic approach and give it formal foundations. To that end, we reinterpret the central judgment of staged logic as *contextual refinement* and rebuild its proof system atop it. A standard proof method for contextual refinement is to construct a *logical relation*; due to the untyped setting used in staged logic, we use the biorthogonal approach to do this, defining mutually recursive relations over both expressions and contexts. We then extend the relation to *meta-contexts* to capture the semantics of the shift and reset operators, a novelty in itself. Finally, we use the logical relation to build a program logic, then a symbolic execution tactic layer on top of it, leading to a practical tool for carrying out refinement proofs of imperative shift/reset programs.

Our contributions are thus:

- A novel biorthogonal binary logical relation for a sequential, imperative, higher-order language with shift/reset (Section 4.2)

- A program logic for establishing contextual refinement for such programs, built on the logical relation, with a symbolic execution layer above, to keep proofs at a high level of abstraction (Section 4.3)
- A complete, sorry-free mechanisation of the framework's metatheory in Lean, and a user interface with tactics for carrying out interactive proofs (Section 4.4)
- An in-depth case study on *control inversion* proved using the framework (Section 5)
- An experimental evaluation on a suite of other case studies, giving an overview of typical proof techniques (Section 6)

2 Motivating Examples

In this section, we look at how to verify shift/reset programs through two examples: we start with a pure program (Section 2.1), where direct reasoning suffices, and then see the challenges that arise when extending this approach to an imperative, heap-manipulating program (Section 2.2), and how our framework addresses them. Our aim is to build intuition for how the framework is used, deferring technical details to later sections. The examples are written in the core language of Section 3, which closely resembles OCaml.

2.1 Verifying a Pure Shift/Reset Program

Our first example is the short-circuiting $times_2$ of Section 1. We prove that it refines the pure *times* it is meant to compute. Fig. 2 restates both programs in the notation of Fig. 5.

$$
\begin{array}{l}
\mathit{let\ rec\ times\ xs} = \\
\quad \mathit{match\ xs\ with} \\
\quad \mid [] \to 1 \\
\quad \mid x :: xs' \to x * \mathit{times}\ xs'
\end{array}
\qquad\qquad
\begin{array}{l}
\mathit{let\ rec\ times}_{sh}\ \mathit{xs} = \\
\quad \mathit{match\ xs\ with} \\
\quad \mid [] \to 1 \\
\quad \mid 0 :: xs' \to \mathit{shift}\ k.\, 0 \\
\quad \mid x :: xs' \to x * \mathit{times}_{sh}\ xs' \\
\mathit{let\ times}_2\ \mathit{xs} = \langle \mathit{times}_{sh}\ \mathit{xs} \rangle
\end{array}
$$

Fig. 2. The pure *times* and the short-circuiting $times_2$

The statement. We want to show that the shift/reset program $times_2$ contextually refines the pure program *times*: in every context, $times_2\ xs$ behaves like $times\ xs$, terminating with the same integer:

$$\forall\, xs.\ \mathit{times}_2\ xs \preceq \mathit{times}\ xs$$

We show two ways to establish it. The first uses a CPS definition, refining $times_2$ into it and it into *times*, and then composes the two by transitivity. The second uses a generalised statement and recovers the original as a corollary, which is more direct but harder to discover. In both attempts, there is no need for a protocol. We prove the refinement by directly reducing the program with its operational semantics, together with an induction on the pure list *xs*.

The CPS definition. Shift and reset are fundamentally connected to the continuation-passing style: they were introduced as a direct-style account of CPS, and every shift/reset program has a meaning-preserving CPS translation [Danvy and Filinski 1990]. This suggests using a CPS definition as an intermediate step for the verification: rather than relating $times_2$ to *times* directly, we define the CPS program $times_{cps}$ of Fig. 3 and factor the refinement through it.

The program $times_{cps}$ receives a continuation k, which is required to be *shift-free* (Fig. 9), i.e., its execution does not invoke shift. In the base case, k is applied to 1. In the recursive case, encountering

a head element of 0 causes $times_{cps}$ to discard k and return 0 directly, corresponding to $times_{sh}$'s abandonment of its continuation via shift. Otherwise, $times_{cps}$ extends k with a multiplication by the head element and recursively processes the remaining list. We now show that the shift/reset program refines this CPS program (SH/CPS), and that the CPS program in turn refines the pure program (CPS/PURE); the original statement follows by transitivity. Both lemmas are proved by induction.

$$
\begin{array}{l}
\mathit{let\ rec}\ \mathit{times}_{cps}\ \mathit{xs}\ k = \\
\quad \mathit{match}\ \mathit{xs}\ \mathit{with} \\
\quad \mid [] \to k\ 1 \\
\quad \mid 0 :: \mathit{xs}' \to 0 \\
\quad \mid x :: \mathit{xs}' \to \mathit{times}_{cps}\ \mathit{xs}'\ (\lambda r.\ k\ (x * r))
\end{array}
$$

SH/CPS

$$\frac{\forall v.\ \mathit{shift\text{-}free}(k\ v)}{\langle \mathit{let}\ r = \mathit{times}_{sh}\ \mathit{xs}\ \mathit{in}\ k\ r\rangle \preceq \mathit{times}_{cps}\ \mathit{xs}\ k}$$

CPS/PURE

$$\frac{0 \preceq k\ 0}{\mathit{times}_{cps}\ \mathit{xs}\ k \preceq \mathit{let}\ r = \mathit{times}\ \mathit{xs}\ \mathit{in}\ k\ r}$$

Fig. 3. The CPS program $times_{cps}$ and the two bridging lemmas.

The premise of SH/CPS requires every application of k to be *shift-free*. Intuitively, *shift-free*(e) says that evaluating e never performs a shift, so the behaviour of e does not depend on any enclosing delimiter. In particular, a reset wrapped around a shift-free expression is redundant and can be removed – a fact we use repeatedly below, in the steps labelled *shift-free*. Section 4 makes this precise as a syntactic judgment; for now, the intuition suffices.

We sketch the proof of SH/CPS, by structural induction on the pure list xs. In the base case where $xs = []$, the goal is discharged by simple evaluation:

$$
\begin{array}{ll}
\quad \langle \mathit{let}\ r = \mathit{times}_{sh}\ []\ \mathit{in}\ k\ r\rangle & \text{unfold } \mathit{times}_{sh} \\
\preceq \langle \mathit{let}\ r = 1\ \mathit{in}\ k\ r\rangle & \text{reduce} \\
\preceq \langle k\ 1\rangle & \text{shift-free} \\
\preceq k\ 1 & \text{fold } \mathit{times}_{cps} \\
\preceq \mathit{times}_{cps}\ []\ k \quad \square &
\end{array}
$$

In the inductive case, the induction hypothesis quantifies over every shift-free continuation, $\forall k'.\ (\forall v.\ \mathit{shift\text{-}free}(k'\ v)) \Rightarrow \langle \mathit{let}\ r = \mathit{times}_{sh}\ \mathit{xs}'\ \mathit{in}\ k'\ r\rangle \preceq \mathit{times}_{cps}\ \mathit{xs}'\ k'$. When the head is 0, the shift body abandons the captured continuation, so the pending multiplications are discarded, and 0 is returned directly:

$$
\begin{array}{ll}
\quad \langle \mathit{let}\ r = \mathit{times}_{sh}\ (0 :: \mathit{xs}')\ \mathit{in}\ k\ r\rangle & \text{unfold } \mathit{times}_{sh} \\
\preceq \langle \mathit{let}\ r = \mathit{shift}\ k.\ 0\ \mathit{in}\ k\ r\rangle & \text{shift-reset} \\
\preceq \langle 0\rangle & \text{shift-free} \\
\preceq 0 & \text{fold } \mathit{times}_{cps} \\
\preceq \mathit{times}_{cps}\ (0 :: \mathit{xs}')\ k \quad \square &
\end{array}
$$

Otherwise the head x is nonzero. We reduce and apply the induction hypothesis:

$$
\begin{array}{ll}
\quad \langle \mathit{let}\ r = \mathit{times}_{sh}\ (x :: \mathit{xs}')\ \mathit{in}\ k\ r\rangle & \text{unfold } \mathit{times}_{sh} \\
\preceq \langle \mathit{let}\ r = x * \mathit{times}_{sh}\ \mathit{xs}'\ \mathit{in}\ k\ r\rangle & \text{reassociate } \mathit{let} \\
\preceq \langle \mathit{let}\ r = \mathit{times}_{sh}\ \mathit{xs}'\ \mathit{in}\ k\ (x * r)\rangle & \text{IH with } k' = \lambda r.\ k\ (x * r) \\
\preceq \mathit{times}_{cps}\ \mathit{xs}'\ (\lambda r.\ k\ (x * r)) & \text{fold } \mathit{times}_{cps} \\
\preceq \mathit{times}_{cps}\ (x :: \mathit{xs}')\ k \quad \square &
\end{array}
$$

The induction hypothesis applies because an application of $k' = \lambda r.\, k\ (x * r)$ is shift-free whenever that of k is: composing a shift-free continuation with pure multiplication preserves shift-freedom. The companion lemma CPS/PURE is proved analogously; its only subtlety is the 0 case, which needs the side condition $0 \preceq k\ 0$ precisely because $times_{cps}$ drops its continuation there while the right-hand side still invokes k. Finally, the original statement follows by transitivity, using the two lemmas above:

$$
\begin{array}{ll}
& times_2\ xs & \text{unfold } times_2 \\
\preceq & \langle times_{sh}\ xs \rangle & \text{let-id} \\
\preceq & \langle let\ r = times_{sh}\ xs\ in\ r \rangle & \text{SH/CPS} \\
\preceq & times_{cps}\ xs\ (\lambda r.\, r) & \text{CPS/PURE} \\
\preceq & let\ r = times\ xs\ in\ r & \text{let-id} \\
\preceq & times\ xs \quad \square &
\end{array}
$$

The generalised statement. We can also prove the statement without the intermediate CPS program, by strengthening it over an accumulator *acc* standing for "the product so far":

$$\forall\, xs\ acc.\ \langle let\ r = times_{sh}\ xs\ in\ r * acc \rangle \preceq times\ xs * acc$$

The original statement follows as a corollary by instantiating $acc = 1$. We prove the generalised statement by structural induction on the pure list *xs*, generalising over *acc*. In the base case $xs = []$, the goal follows directly by evaluation:

$$
\begin{array}{ll}
& \langle let\ r = times_{sh}\ []\ in\ r * acc \rangle & \text{unfold } times_{sh} \\
\preceq & \langle let\ r = 1\ in\ r * acc \rangle & \text{reduce} \\
\preceq & \langle 1 * acc \rangle & \text{shift-free} \\
\preceq & 1 * acc & \text{fold } times \\
\preceq & times\ [] * acc \quad \square &
\end{array}
$$

In the inductive case, the induction hypothesis is generalised over the accumulator: $\forall\, acc'.\ \langle let\ r = times_{sh}\ xs'\ in\ r * acc' \rangle \preceq times\ xs' * acc'$. We proceed by case analysis on the head of the list. When the head is 0, the left-hand side uses shift to abandon the captured continuation and return 0 directly. On the right-hand side, the result is also 0 by the absorbing property of multiplication:

$$
\begin{array}{ll}
& \langle let\ r = times_{sh}\ (0 :: xs')\ in\ r * acc \rangle & \text{unfold } times_{sh} \\
\preceq & \langle let\ r = shift\ k.\, 0\ in\ r * acc \rangle & \text{shift-reset} \\
\preceq & \langle 0 \rangle & \text{shift-free} \\
\preceq & 0 & \text{arithmetic} \\
\preceq & 0 * times\ xs' * acc & \text{fold } times \\
\preceq & times\ (0 :: xs') * acc \quad \square &
\end{array}
$$

When the head is non-zero, we reduce and apply the induction hypothesis:

$$
\begin{array}{ll}
& \langle let\ r = times_{sh}\ (x :: xs')\ in\ r * acc \rangle & \text{unfold } times_{sh} \\
\preceq & \langle let\ r = x * times_{sh}\ xs'\ in\ r * acc \rangle & \text{reassociate } let \\
\preceq & \langle let\ r = times_{sh}\ xs'\ in\ r * x * acc \rangle & \text{IH with } acc' = x * acc \\
\preceq & times\ xs' * x * acc & \text{arithmetic} \\
\preceq & x * times\ xs' * acc & \text{fold } times \\
\preceq & times\ (x :: xs') * acc \quad \square &
\end{array}
$$

2.2 Verifying an Imperative Shift/Reset Program

Our second example combines delimited control with mutable state: the programs of Fig. 4 both use shift/reset and manipulate the heap, and their correctness hinges on the interaction of the two. *flip* receives a reference *x*; it captures its continuation with shift, resumes it twice – once with *true* and

once with *false* – and returns the sum of the two results. Because each resumption first increments the integer stored at x, the reference ends up counting how many times the continuation is invoked. flip_c runs *flip* under a delimiter and converts *true* into 1 and *false* into 0. *toss n x* calls *flip n* times on x and returns the conjunction of the n boolean results, while toss_c plays the same wrapping role for *toss* as flip_c does for *flip*.

```
let flip x =
  shift k. let _ = x := !x + 1 in
           let r1 = k true in
           let _ = x := !x + 1 in
           let r2 = k false in
           r1 + r2
let flip_c v =
  let x = ref v in
  let r = ⟨if flip x then 1 else 0⟩ in
  (r, !x)
```

```
let rec toss n x =
  if n = 0 then true else
    let r1 = flip x in
    let r2 = toss (n − 1) x in r1 && r2

let toss_c n v =
  let x = ref v in
  let r = ⟨if toss n x then 1 else 0⟩ in
  (r, !x)
```

Fig. 4. flip_c and toss_c, combining shift/reset with a mutable reference

The reference x is private to flip_c: it is allocated before the delimiter, with caller-chosen initial contents v, and never escapes. To let the caller observe what happened to x, flip_c returns the result of the reset paired with the final contents of x, read only after the reset has returned. Because *flip* resumes the captured continuation twice, each resumption increments x once and produces 1 and 0 respectively, so the reset returns their sum 1 while x ends up holding $v + 2$. We state this as a refinement of the pure pair $(1, v + 2)$:

$$\forall\, v.\ \mathit{flip}_c\ v \preceq (1, v + 2)$$

toss_c follows the same pattern – x is again private, allocated before the delimiter with initial contents v and read back only after the reset has returned – but its behaviour is trickier to pin down. Performing n flips unfolds a binary decision tree of height n: each flip resumes its continuation twice, so x is incremented once per edge, and a tree of height n has $2^{n+1}-2$ edges. The reset, meanwhile, returns 1: each leaf contributes 1 if every flip along its path produced *true* and 0 otherwise, the flips sum both branches at every node, and exactly one of the 2^n leaves is all-*true*. Since *toss* recurses on $n - 1$, a negative n would cause it to diverge; here and below, we therefore assume n is a natural number. We state this as a refinement:

$$\forall\, n\, v.\ \mathit{toss}_c\ n\ v \preceq (1, v + 2^{n+1}-2)$$

We prove this refinement by induction on n. However, unlike for times_2, working with contextual refinement directly is impossible here: runtime locations, which arise only during evaluation, are not allowed in the statement of a contextual refinement. Yet once toss_c allocates x, the intermediate goals of the induction are exactly programs containing the runtime location x, and their behaviour depends on what x currently holds. Contextual refinement gives us no way to state such goals, let alone to track the contents of x as we descend through the recursive calls to *toss*.

We instead switch to the program logic of Section 4.3, whose judgment $\{\Phi\}\ e \preceq e'\ \{\Phi'\}$ addresses both problems: runtime locations may appear in the related programs, and what they hold is recorded in the heap assertions Φ and Φ', in the style of separation logic. Using its soundness rule,

a derivation in the logic yields a contextual refinement: to prove the statement above, it suffices to derive:

$$\{emp\}\ toss_c\ n\ v \preceq (1, v + 2^{n+1} - 2)\ \{\Phi'\}$$

for some postcondition Φ'.

We symbolically execute the left-hand side one step at a time. The first imperative step is the allocation of x: executing it introduces a fresh location x together with the points-to assertion $x \mapsto v$ recording its contents, and leaves the rest of the program to be related to the same right-hand side under this precondition:

$$\{x \mapsto v\}\ let\ r = \langle if\ \ toss\ n\ x\ then\ 1\ else\ 0\rangle\ in\ (r, !x) \preceq (1, v + 2^{n+1} - 2)\ \{\Phi'\}$$

A direct induction on this goal gets stuck: after executing a flip, the goal no longer matches the original statement – the reset body has grown a pending conjunction, and the contents of x have changed. We therefore strengthen the statement, generalising over the current contents v of x and adding an accumulator *acc* for the conjunction of the flip results gathered so far. Focusing on the reset expression, we will now prove:

$$\forall\, n\, x\, v\, acc.\ \{x \mapsto v\}\ \langle let\ r = toss\ n\ x\ in\ if\ \ acc\ \&\&\ r\ then\ 1\ else\ 0\rangle \\ \preceq if\ \ acc\ then\ 1\ else\ 0\ \{x \mapsto v + 2^{n+1} - 2\}$$

We prove it by induction on n, with x fixed and v, *acc* generalised. In the base case where $n = 0$, $toss\ 0\ x$ returns *true* without modifying x. The goal is discharged by evaluation:

$$\begin{array}{ll} & \{x \mapsto v\} \\ \preceq & \langle let\ r = toss\ 0\ x\ in\ if\ \ acc\ \&\&\ r\ then\ 1\ else\ 0\rangle \quad \text{reduce, shift-free} \\ & \{x \mapsto v\} \\ \preceq & if\ \ acc\ then\ 1\ else\ 0 \\ & \{x \mapsto v\} \quad \square \end{array}$$

and $v = v + 2^{0+1} - 2$, matching the goal. In the inductive case where $n = n' + 1$, the induction hypothesis is the statement at n', generalised over the contents of x and the accumulator:

$$\forall\, v'\ acc'.\ \{x \mapsto v'\}\ \langle let\ r = toss\ n'\ x\ in\ if\ \ acc'\ \&\&\ r\ then\ 1\ else\ 0\rangle \\ \preceq if\ \ acc'\ then\ 1\ else\ 0\ \{x \mapsto v' + 2^{n'+1} - 2\}$$

We reduce by unfolding *toss* and *flip* and applying shift-reset, where the shift inside *flip* captures the following continuation:

$$k = \lambda b.\ \langle let\ r = toss\ n'\ x\ in\ if\ \ acc\ \&\&\ (b\ \&\&\ r)\ then\ 1\ else\ 0\rangle$$

We chain the following steps, with a heap assertion between them:

$$
\begin{array}{ll}
& \{x \mapsto v\} \\
\preceq & \langle \mathit{let}\ r = \mathit{toss}\ (n'+1)\ x\ \mathit{in}\ \mathit{if}\ \mathit{acc}\ \&\&\ r\ \mathit{then}\ 1\ \mathit{else}\ 0\rangle \quad \text{reduce, shift-free} \\
& \{x \mapsto v\} \\
\preceq & \mathit{let}\ _ = x := !x + 1\ \mathit{in}\ \mathit{let}\ r_1 = k\ \mathit{true}\ \mathit{in} \quad \text{store} \\
& \mathit{let}\ _ = x := !x + 1\ \mathit{in}\ \mathit{let}\ r_2 = k\ \mathit{false}\ \mathit{in}\ r_1 + r_2 \\
& \{x \mapsto v + 1\} \\
\preceq & \mathit{let}\ r_1 = k\ \mathit{true}\ \mathit{in} \quad \text{IH with } v' = v+1,\ \mathit{acc}' = \mathit{acc} \\
& \mathit{let}\ _ = x := !x + 1\ \mathit{in}\ \mathit{let}\ r_2 = k\ \mathit{false}\ \mathit{in}\ r_1 + r_2 \\
& \{x \mapsto v + 2^{n'+1} - 1\} \\
\preceq & \mathit{let}\ _ = x := !x + 1\ \mathit{in} \quad \text{store} \\
& \mathit{let}\ r_2 = k\ \mathit{false}\ \mathit{in}\ (\mathit{if}\ \mathit{acc}\ \mathit{then}\ 1\ \mathit{else}\ 0) + r_2 \\
& \{x \mapsto v + 2^{n'+1}\} \\
\preceq & \mathit{let}\ r_2 = k\ \mathit{false}\ \mathit{in}\ (\mathit{if}\ \mathit{acc}\ \mathit{then}\ 1\ \mathit{else}\ 0) + r_2 \quad \text{IH with } v' = v + 2^{n'+1},\ \mathit{acc}' = \mathit{false} \\
& \{x \mapsto v + 2^{n'+2} - 2\} \\
\preceq & \mathit{if}\ \mathit{acc}\ \mathit{then}\ 1\ \mathit{else}\ 0 \\
& \{x \mapsto v + 2^{n'+2} - 2\} \quad \square
\end{array}
$$

and $2^{n'+2} - 2 = 2^{n+1} - 2$ as $n = n' + 1$, matching the goal.

Finally, instantiating the generalised statement with $\mathit{acc} = \mathit{true}$ and symbolically executing what remains – the dereference of x and the construction of the pair – discharges the goal for toss_c, completing the proof.

3 Core Language

$$
\begin{array}{rl}
e ::= & v \mid x \mid e\ e \\
& \mid \mathit{let}\ x = e_1\ \mathit{in}\ e_2 \\
& \mid \mathit{ref}\ e \mid\ !e \mid e := e \\
& \mid \mathit{shift}\ k.\, e \mid \langle e \rangle \\
v, w, f ::= & () \mid \ell \mid \lambda x.\, e
\end{array}
\qquad
\begin{array}{rl}
K ::= & \square \mid K\ e \mid v\ K \\
& \mid \mathit{let}\ x = K\ \mathit{in}\ e_2 \\
& \mid \mathit{ref}\ K \mid\ !K \mid K := e \mid v := K \\
M ::= & \epsilon \mid K \cdot M
\end{array}
$$

Fig. 5. Syntax of the core language

Syntax. We work with a core language, an untyped, deterministic subset of OCaml extended with the delimited control operators shift and reset; its syntax is given in Fig. 5. Values v comprise the unit value, runtime locations ℓ, and functions. Expressions e include values, variables, function application, and let-bindings, together with the heap primitives: allocation $\mathit{ref}\ e$, dereference $!e$, and assignment $e_1 := e_2$. Finally, $\mathit{shift}\ k.\, e$ captures the current delimited continuation, and $\langle e \rangle$ delimits it. The language also has other standard features – base literals (integers, booleans, strings) with their primitive operations, pairs, recursive functions, conditionals, and algebraic data types with pattern matching – which we use freely in examples but omit from the figures, as their treatment is entirely standard. Locations arise only at runtime, as the result of allocation, and do not appear in source programs. Throughout, writing a term with a value metavariable (such as v, w, or f) signals that it is required or guaranteed to be a value.

Semantics. The semantics is given in a small-step contextual style, using an evaluation context K and a meta-context M, following the operational account of delimited continuations of Biernacka

$\boxed{(h,\ e) \rightsquigarrow (h',\ e')}$

STEP-BETA
$$(h,\ (\lambda x.\, e)\ v) \rightsquigarrow (h,\ e[v/x])$$

STEP-LET
$$(h,\ let\ x = v\ in\ e) \rightsquigarrow (h,\ e[v/x])$$

STEP-RESET
$$(h,\ \langle v \rangle) \rightsquigarrow (h,\ v)$$

STEP-ALLOC
$$\frac{\ell \notin dom(h)}{(h,\ ref\ v) \rightsquigarrow (h[\ell \mapsto v],\ \ell)}$$

STEP-LOAD
$$\frac{h(\ell) = v}{(h,\ !\ell) \rightsquigarrow (h,\ v)}$$

STEP-STORE
$$\frac{\ell \in dom(h)}{(h,\ \ell := v) \rightsquigarrow (h[\ell \mapsto v],\ ())}$$

$\boxed{(h,\ e) \hookrightarrow (h',\ e')}$

STEP-CONTEXT
$$\frac{(h,\ e) \rightsquigarrow (h',\ e')}{(h,\ K[e]) \hookrightarrow (h',\ K[e'])}$$

STEP-SHIFT
$$(h,\ K[shift\ k.\, e]) \hookrightarrow (h,\ e[\lambda x.\, \langle K[x] \rangle / k])$$

$\boxed{(h,\ e) \longrightarrow (h',\ e')}$

STEP-META-CONTEXT
$$\frac{(h,\ e) \hookrightarrow (h',\ e')}{(h,\ M[e]) \longrightarrow (h',\ M[e'])}$$

$$\epsilon[e] \triangleq e \qquad (K \cdot M)[e] \triangleq M[K[\langle e \rangle]]$$

Fig. 6. Small-step contextual semantics of the core language

et al. [2005]. It is presented in Fig. 6 as three layers. Configurations c are pairs $(h,\ e)$ of a heap h and an expression e.

The *base step* $(h,\ e) \rightsquigarrow (h',\ e')$ applies when e is itself a redex and contracts it: β-reduction for functions, unfolding of *let*, the heap primitives, and the removal of a delimiter around a value ($\langle v \rangle \rightsquigarrow v$). The evaluation context K fixes a left-to-right, call-by-value order. Crucially, K contains no reset, and thus will always be inside an enclosing reset. The *contextual step* $(h,\ e) \hookrightarrow (h',\ e')$ lifts a base step under an evaluation context K and captures the continuation on shift. The meta-context M is a stack of evaluation contexts, one frame per enclosing reset; plugging $M[e]$ re-installs a $\langle\cdot\rangle$ at each frame. The *meta-contextual step* $(h,\ e) \longrightarrow (h',\ e')$ then lets a contextual step fire underneath the delimiters installed by M. Evaluation of a whole program e is the reflexive-transitive closure $\longrightarrow^*$ starting from $(\emptyset,\ e)$, under an empty heap.

A shift with no enclosing $\langle\cdot\rangle$ is treated as a shift with an enclosing $\langle\cdot\rangle$ at the top level. We make this choice to stay faithful to the presentation of Biernacka et al. [2005]. Restricting shift to be stuck without an enclosing delimiter is a simple change and will not be considered in this paper.

Capturing a continuation. When evaluation reaches $K[shift\ k.\, e]$, rule STEP-SHIFT captures the current evaluation context K – everything up to the innermost enclosing reset – and reifies it as the ordinary function value $\lambda x.\, \langle K[x] \rangle$, which is substituted for k in the shift body e. There is no separate syntactic class of continuations: a captured continuation is simply a λ-abstraction.

As is standard for shift/reset, both performing a shift and invoking a captured continuation re-install a delimiter: the enclosing reset is not consumed by the capture, so the shift body e continues to run under the same delimiter, and the reified continuation wraps its context in a fresh $\langle\cdot\rangle$, so its body also runs under a delimiter when invoked.

4 Program Logic

In this section, we construct a program logic in increments, starting with preliminaries on contextual refinement, the property we are interested in, and a proof system usable for pure shift/reset programs (Section 4.1); then, a logical relation, a proof method for contextual refinement (Section 4.2), and a step on the way to the program logic; then the program logic and its various judgments, along with mechanisms for symbolic execution (Section 4.3); and finally some proof engineering aspects to turn the program logic into a practical verification tool (Section 4.4).

4.1 Contextual Refinement

We begin with the notion of general program contexts, and what we can observe about programs.

Program contexts. A *program context* C is a program with a single hole $\Box$, where the hole may occur under binders (Fig. 7).

$$\begin{aligned} C ::= \Box \mid \lambda x.\, C \mid C\ e \mid e\ C \mid \mathit{let}\ x = C\ \mathit{in}\ e \mid \mathit{let}\ x = e\ \mathit{in}\ C \\ \mid \mathit{ref}\ C \mid\ !C \mid C := e \mid e := C \mid \mathit{shift}\ k.\, C \mid \langle C \rangle \end{aligned}$$

Fig. 7. Syntax of program contexts C

We write $C[e]$ for the program obtained by filling the hole with e, potentially capturing free variables of e. A context is *closing* for e when $C[e]$ binds all free variables of e, yielding a closed program. We further require the context to be *location-free*: it contains no runtime locations ℓ, which arise only during evaluation and never in a source program.

Observations. The only observation we make of a closed program is whether it terminates. $e \Downarrow$ denotes that e, run under the empty heap, reduces to a value (Fig. 6).

$$e \Downarrow \triangleq \exists h, v. (\emptyset, e) \longrightarrow^* (h, v)$$

Definition 4.1 (Contextual refinement and equivalence)**.** *For programs e_1 and e_2,*

$$\begin{aligned} e_1 \preceq e_2 &\triangleq \forall C.\ \ C\ \mathit{closing} \Rightarrow C[e_1] \Downarrow \ \Rightarrow\ C[e_2] \Downarrow \\ e_1 \approx e_2 &\triangleq e_1 \preceq e_2 \wedge e_2 \preceq e_1 \end{aligned}$$

$e_1 \preceq e_2$ means that whenever a context C makes e_1 terminate, C also makes e_2 terminate. $\approx$ is the induced equivalence.

Fig. 8 gives a proof system for pure programs with shift and reset, whose control laws are inspired by the sound and complete axiomatization of Kameyama and Hasegawa [2003]. Contextual refinement is a precongruence: reflexive and transitive, and a congruence, so contextual equivalence rules may be used to rewrite subterms on either side of a contextual refinement freely. The functional (E-LET, E-BETA, E-ETA) and monadic rules (E-LET-ASSOC, E-LET-ID) are standard contextual equivalences. The last three rules pertain to shift and reset. E-SHIFT-RESET internalises the STEP-SHIFT rule of the operational semantics. E-SHIFT-FREE-RESET removes a redundant delimiter whenever e is *shift-free*; we make this notion precise as the syntactic judgment *shift-free*(e) (Fig. 9). E-RESET-LET [Kameyama 2004] allows adding a delimiter to the body of a *let*.

Values, including λ-abstractions, are shift-free (SF-VAL), but an application of a λ-abstraction is shift-free only when the body itself is (SF-APP), since reducing the redex puts that body in evaluation position. A reset is shift-free (SF-RESET) regardless of its contents: any shift inside is captured by that inner delimiter and cannot escape. A bare shift has no rule, so it is never shift-free. The remaining rules simply propagate shift-freedom to the subexpressions.

$$\textsc{refl}\ \ e \preceq e \qquad \textsc{trans}\ \ \frac{e_1 \preceq e_2 \qquad e_2 \preceq e_3}{e_1 \preceq e_3} \qquad \textsc{cong}\ \ \frac{e_1 \preceq e_2}{C[e_1] \preceq C[e_2]}$$

$$\textsc{e-beta}\ \ (\lambda x.\, e)\ v \approx e[v/x] \qquad \textsc{e-eta}\ \ \frac{x \notin FV(e)}{\lambda x.\, (\lambda y.\, e)\ x \approx \lambda y.\, e} \qquad \textsc{e-let}\ \ \mathit{let}\ x = v\ \mathit{in}\ e \approx e[v/x]$$

$$\textsc{e-let-assoc}\ \ \mathit{let}\ y = (\mathit{let}\ x = e_1\ \mathit{in}\ e_2)\ \mathit{in}\ e_3 \approx \mathit{let}\ x = e_1\ \mathit{in}\ (\mathit{let}\ y = e_2\ \mathit{in}\ e_3) \qquad \textsc{e-let-id}\ \ \mathit{let}\ x = e\ \mathit{in}\ x \approx e$$

$$\textsc{e-shift-reset}\ \ \langle K[\mathit{shift}\ k.\, e]\rangle \approx \langle e[\lambda x.\, \langle K[x]\rangle / k]\rangle \qquad \textsc{e-shift-free-reset}\ \ \frac{\mathit{shift\text{-}free}(e)}{\langle e\rangle \approx e} \qquad \textsc{e-reset-let}\ \ \langle \mathit{let}\ x = e_1\ \mathit{in}\ e_2\rangle \approx \langle \mathit{let}\ x = e_1\ \mathit{in}\ \langle e_2\rangle\rangle$$

Fig. 8. Select rules for contextual refinement.

$$\boxed{\mathit{shift\text{-}free}(e)}$$

$$\textsc{sf-val}\ \ \frac{}{\mathit{shift\text{-}free}(v)} \qquad \textsc{sf-var}\ \ \frac{}{\mathit{shift\text{-}free}(x)} \qquad \textsc{sf-reset}\ \ \frac{}{\mathit{shift\text{-}free}(\langle e\rangle)} \qquad \textsc{sf-app}\ \ \frac{\mathit{shift\text{-}free}(e_1) \qquad \mathit{shift\text{-}free}(e_2)}{\mathit{shift\text{-}free}((\lambda x.\, e_1)\ e_2)}$$

$$\textsc{sf-let}\ \ \frac{\mathit{shift\text{-}free}(e_1) \qquad \mathit{shift\text{-}free}(e_2)}{\mathit{shift\text{-}free}(\mathit{let}\ x = e_1\ \mathit{in}\ e_2)} \qquad \textsc{sf-ref}\ \ \frac{\mathit{shift\text{-}free}(e)}{\mathit{shift\text{-}free}(\mathit{ref}\ e)} \qquad \textsc{sf-deref}\ \ \frac{\mathit{shift\text{-}free}(e)}{\mathit{shift\text{-}free}(!e)} \qquad \textsc{sf-assign}\ \ \frac{\mathit{shift\text{-}free}(e_1) \qquad \mathit{shift\text{-}free}(e_2)}{\mathit{shift\text{-}free}(e_1 := e_2)}$$

Fig. 9. Shift-freedom.

Pure shift/reset programs can be reasoned about entirely using this simple set of rules. Two problems remain: firstly, there are no rules for heap operations, and secondly, there is the matter of actually *establishing* these rules and showing that they are sound.

4.2 Logical Relation

The universal quantification over all closing contexts makes contextual refinement hard to establish directly [Balik et al. 2025]. The standard proof method for it is a (binary) *logical relation*, a relation between two programs typically defined by recursion on their structure. Because it is sound with respect to contextual refinement, we can use it to establish the latter by reasoning about a program's behaviour directly, rather than about contexts.

We define a biorthogonal, step-indexed, Kripke logical relation, for the full language of Fig. 5, which is untyped. This will be unpacked shortly. Its novel ingredient is the relation $\mathcal{M}$, which relates meta-contexts, extending the technique to delimited control.

Step-indexing. We first assume a base logic with a *later modality* $\triangleright$ [Dreyer et al. 2011], a logical device for constructing a set of mutually recursive definitions which are not structurally well-founded, and giving us an induction principle for them, *Löb induction*. The later modality internalizes

the notion of a *step*, which decreases at recursive calls and recovers well-foundedness; in program logics, these steps typically correspond to execution steps.

The logic we use here is the logic of *hereditary predicates* from *Indirection Theory* [Hobor et al. 2010]. Due to being *step-indexed* [Appel and McAllester 2001], the predicates of this logic are *approximations* whose truth values are only guaranteed for a finite number of steps, though quantification allows reasoning about properties arbitrarily far into the future. It also makes available a concept of *worlds*, which may be thought of in our setting as finite maps that may contain hereditary predicates. Worlds may be *aged*, which weakens the approximations of their contents by decreasing the step-index. Hereditary predicates are so named because they have truth values which are preserved under aging.

Configurations and observations. As we are now concerned with stateful programs, we talk about their execution using *configurations* $c ::= (h,\ e)$ of a heap and an expression. Again, we only observe termination of configurations: $(h,\ e)\Downarrow$ holds when $(h,\ e) \longrightarrow^* (h',\ v)$ for some heap h' and value v. The top-level definition $e \Downarrow$ of Section 4.1 is equivalent to $(\emptyset,\ e)\Downarrow$.

At the base of the construction is the forward observation relation $\mathcal{L}$ (Fig. 10): $(c_1, c_2) \in \mathcal{L}(W)$ says that c_1 refines c_2: if c_1 evaluates to a value, c_2 terminates, otherwise, every successor c_1' of c_1 continues to refine c_2 *one step later*. "Taking the limit" of this guarded definition (by Löb induction) yields $c_1 \Downarrow \Rightarrow c_2 \Downarrow$. $\mathcal{O}$ (Fig. 10) is $\mathcal{L}$ in both directions.

Worlds and heaps. Here, we use a world W as a finite map, to maintain a partial bijection between the locations of the two heaps of our configurations. We write $\ell_1 \sim_W \ell_2$ when W ties ℓ_1 on the left to ℓ_2 on the right, i.e., $W(\ell_1) = \ell_2$. Worlds only ever grow: *world extension* $W' \sqsupseteq W$ means W' extends the bijection of W. Two heaps *satisfy* a world, written $h_1 \bowtie_W h_2$, when W is a bijection between $dom(h_1)$ and $dom(h_2)$ and every tied pair $\ell_1 \sim_W \ell_2$ stores $\mathcal{V}$-related values ($\mathcal{V}$ is defined next).

$$
\begin{aligned}
(c_1, c_2) \in \mathcal{L}(W) &\triangleq (c_1 \text{ a value config} \Rightarrow c_2 \Downarrow) \ \wedge\ (\forall\, c_1'.\ c_1 \longrightarrow c_1' \Rightarrow \triangleright\, (c_1', c_2) \in \mathcal{L}(W)) \\
(c_1, c_2) \in \mathcal{O}(W) &\triangleq (c_1, c_2) \in \mathcal{L}(W) \ \wedge\ (c_2, c_1) \in \mathcal{L}(W) \\
(v_1, v_2) \in \mathcal{V}(W) &\triangleq (v_1, v_2) = ((), ()) \\
&\quad \mid (v_1, v_2) = (\ell_1, \ell_2) \ \wedge\ \ell_1 \sim_W \ell_2 \\
&\quad \mid (v_1, v_2) = (\lambda x.\, e_1, \lambda x.\, e_2) \ \wedge\ \forall\, W' \sqsupseteq W.\ \forall\, (u_1, u_2) \in \triangleright \mathcal{V}(W').\ (v_1\ u_1, v_2\ u_2) \in \mathcal{R}(W') \\
(M_1, M_2) \in \mathcal{M}(W) &\triangleq \forall\, W' \sqsupseteq W.\ \forall\, (v_1, v_2) \in \mathcal{V}(W').\ \forall\, h_1 \bowtie_{W'} h_2. \\
&\qquad ((h_1,\ M_1[v_1]),\ (h_2,\ M_2[v_2])) \in \mathcal{L}(W') \\
(K_1, K_2) \in \mathcal{K}(W) &\triangleq \forall\, W' \sqsupseteq W.\ \forall\, (M_1, M_2) \in \mathcal{M}(W').\ \forall\, (v_1, v_2) \in \mathcal{V}(W').\ \forall\, h_1 \bowtie_{W'} h_2. \\
&\qquad ((h_1,\ M_1[K_1[v_1]]),\ (h_2,\ M_2[K_2[v_2]])) \in \mathcal{L}(W') \\
(e_1, e_2) \in \mathcal{R}(W) &\triangleq \forall\, (M_1, M_2) \in \triangleright \mathcal{M}(W).\ \forall\, (K_1, K_2) \in \triangleright \mathcal{K}(W).\ \forall\, h_1 \bowtie_W h_2. \\
&\qquad ((h_1,\ M_1[K_1[e_1]]),\ (h_2,\ M_2[K_2[e_2]])) \in \mathcal{L}(W) \\
(e_1, e_2) \in \mathcal{E}(W) &\triangleq \forall\, W' \sqsupseteq W.\ \forall\, (M_1, M_2) \in \mathcal{M}(W').\ \forall\, (K_1, K_2) \in \mathcal{K}(W').\ \forall\, h_1 \bowtie_{W'} h_2. \\
&\qquad ((h_1,\ M_1[K_1[e_1]]),\ (h_2,\ M_2[K_2[e_2]])) \in \mathcal{L}(W')
\end{aligned}
$$

Fig. 10. The biorthogonal, step-indexed, Kripke logical relation.

The logical relation. The rest of Fig. 10 defines a mutually recursive family of relations. They are *Kripke* logical relations because they are indexed by worlds (with their extension relation),

$\mathcal{V}$ relates values. Unit values are related; location values are related exactly when the world ties them; and two functions are related when, at every future world and on every pair of $\triangleright$-related value arguments, their applications are expressions related by $\mathcal{R}$. $\triangleright$ makes the negative recursive

occurrence of $\mathcal{V}$ well-founded. The clauses for the standard constructs elided from Fig. 5 are as expected: base literals are related exactly when they are equal, pairs are related componentwise, and recursive functions are related as for functions. Equality of base literals is what lets refinements state concrete results, as in $\forall v.\ \mathit{flip}_c\ v \preceq (1, v+2)$ (Section 2.2).

Next, the expression relations $\mathcal{R}$ and $\mathcal{E}$ relate two expressions. They quantify over the contexts and meta-contexts that the expressions are plugged in to yield the observation; these are assumed to be related by $\mathcal{K}$ and $\mathcal{M}$ (next paragraph) in a mutually recursive way. They differ only in guardedness: $\mathcal{R}$ requires the surrounding $\mathcal{M}$ and $\mathcal{K}$ only *later* ($\triangleright$), and this is the form used inside $\mathcal{V}$'s function clause, whereas $\mathcal{E}$ is the unguarded relation used everywhere else.

Last are the context and meta-context relations $\mathcal{K}$ and $\mathcal{M}$. A *meta-context* pair (a stack of delimiters, one per enclosing reset) is in $\mathcal{M}$ when it maps related values, under satisfying heaps, to $\mathcal{L}$-related configurations. A(n evaluation) *context* pair is in $\mathcal{K}$ when, plugged with related values and plugged into related meta-contexts, it produces $\mathcal{L}$-related configurations.

Open programs, the fundamental theorem, and soundness. The relations above are on closed programs; open programs are related under related substitutions. Writing γ_1, γ_2 for substitutions with $(\gamma_1(x), \gamma_2(x)) \in \mathcal{V}(W)$ for every free variable x, we define

$$e_1 \preceq_{\log} e_2 \triangleq \forall W.\ \forall \gamma_1, \gamma_2.\ (\gamma_1\, e_1,\ \gamma_2\, e_2) \in \mathcal{E}(W),$$

the *logical refinement* between (possibly open) programs. The *fundamental theorem* of the logical relation says every source program is logically related to itself, which is what makes $\preceq_{\log}$ a congruence and reduces each refinement rule to a local check.

Theorem 4.1 (Fundamental theorem). *For every location-free term e, $e \preceq_{\log} e$.* □

It is proved by a set of compatibility lemmas, one per constructor of Fig. 5. Soundness then lifts the logical relation to contextual refinement.

Theorem 4.2 (Soundness). *If $e_1 \preceq_{\log} e_2$ then $e_1 \preceq e_2$.* □

Soundness is proved by instantiating $\preceq_{\log}$ with the empty world and the identity substitution, observing at the empty heap, and closing $\mathcal{L}$ under an arbitrary program context using the fundamental theorem. This way, a logical refinement between e_1 and e_2 forces termination to be preserved in every closing context, which is exactly $e_1 \preceq e_2$. Many of the rules of Section 4.1, particularly the reduction ones, are established by exhibiting a witness in the logical relation.

4.3 Symbolic Execution

The logical relation of Section 4.2 is an adequate proof method for contextual refinement. However, using it directly, by unfolding its definition and reasoning operationally, would be extremely laborious, and a nonstarter for larger case studies. Hence, we backtrack a little, and instead use the ingredients introduced to build a program logic for proving contextual refinement compositionally, in a way that supports symbolic execution of the programs. This keeps reasoning at a high level of abstraction.

The main judgment has the shape $\{\Phi\}\ e \preceq e'\ \{\Phi'\}$, a relational triple specialised to contextual refinement. We sometimes refer to e as the *implementation* side and e' as the *specification* side.

Heap assertions. The specifications of our program logic are *heap assertions* Φ, world-indexed relational pre- and postconditions over two heaps (Fig. 11). *emp* imposes no constraints. Instead of a single *points-to* assertion, there are two single-sided *private* assertions which constrain only one of two heaps. $\ell \mapsto v$ says that ℓ holds v in the *left* (implementation) heap and is *untied*: related to no right-side location by the world W. $\ell \mapsto_s v$ (s for *specification*) similarly constrains only

the right heap. Untiedness is what allows single-sided mutation: a private location on, e.g., the implementation side cannot be observed by the specification program, regardless of context, so mutating it requires no matching step on the right. $\Phi \star \Psi$ is separating conjunction, lifted pointwise to two heaps. Heap assertions are *hereditary* (*Hered* Φ) when their truth value is preserved by aging of worlds, which occurs on pure steps; all assertions presented thus far are hereditary. Finally, heap assertions are ordered by *heap entailment* $\Phi \vdash \Psi$, which lifts implication pointwise to heap assertions.

$$
\begin{array}{rcl}
\Phi & : & \mathit{Heap} \times \mathit{Heap} \times \mathit{World} \to \mathit{Prop} \\
\mathit{emp} & \triangleq & \lambda(h_1, h_2, W).\ \mathit{True} \\
\ell \mapsto v & \triangleq & \lambda(h_1, h_2, W).\ h_1(\ell) = v \ \wedge\ \forall \ell'.\ \neg\, \ell \sim_W \ell' \\
\ell \mapsto_s v & \triangleq & \lambda(h_1, h_2, W).\ h_2(\ell) = v \ \wedge\ \forall \ell'.\ \neg\, \ell' \sim_W \ell \\
\Phi \star \Psi & \triangleq & \lambda(h_1, h_2, W).\ \exists\, h_1^l \uplus h_1^r = h_1,\ h_2^l \uplus h_2^r = h_2.\ \Phi\,(h_1^l, h_2^l, W) \wedge \Psi\,(h_1^r, h_2^r, W) \\
\mathit{Hered}\ \Phi & \triangleq & \forall\, W' \sqsupseteq W.\ \Phi\,(h_1, h_2, W) \Rightarrow \Phi\,(h_1, h_2, W') \\
\Phi \vdash \Psi & \triangleq & \forall\, h_1, h_2, W.\ \Phi\,(h_1, h_2, W) \Rightarrow \Psi\,(h_1, h_2, W)
\end{array}
$$

Fig. 11. Heap assertions: relational pre/postconditions over two heaps.

Refinement under a precondition. Next, we define the main judgment, and an alternative one which illustrates different tradeoffs. Both are defined in terms of a more primitive one, $\Phi \vdash e_1 \preceq e_2$ (pronounced *refines under* Φ; Fig. 12), which is effectively the $\mathcal{E}$ relation under a precondition. In short, e_1 and e_2 must be in the forward observation $\mathcal{L}$, for all meta-contexts, contexts, and tied heaps satisfying the precondition Φ. It is adequate with respect to contextual refinement: $\mathit{emp} \vdash e_1 \preceq e_2 \Rightarrow e_1 \preceq e_2$ holds. It can be treated abstractly from here on.

$$
\begin{array}{rcl}
\Phi \vdash e_1 \preceq e_2 & \triangleq & \forall\, W.\ \forall\,(M_1, M_2) \in \mathcal{M}(W).\ \forall\,(K_1, K_2) \in \mathcal{K}(W). \\
& & \quad \forall\, h_1 \bowtie_W h_2.\ \Phi\,(h_1, h_2, W) \Rightarrow \\
& & \quad\quad ((h_1,\ M_1[K_1[e_1]]),\ (h_2,\ M_2[K_2[e_2]])) \in \mathcal{L}(W)
\end{array}
$$

$$
\begin{array}{rcl}
D & ::= & K \mid K[\langle D \rangle] \\
\{\Phi\}\ e_1 \preceq e_2\ \{\Phi'\} & \triangleq & \forall\, D\, b.\ \Phi' \vdash D[e_2] \preceq b \Rightarrow \Phi \vdash D[e_1] \preceq b
\end{array}
$$

L-BIND

$$\frac{\mathit{Hered}\ \Phi_m \qquad \{\Phi\}\ e \preceq v\ \{\Phi_m\} \qquad \{\Phi_m\}\ D[v] \preceq e'\ \{\Phi'\}}{\{\Phi\}\ D[e] \preceq e'\ \{\Phi'\}}$$

L-SOUND

$$\frac{\{\mathit{emp}\}\ e_1 \preceq e_2\ \{\Phi\} \qquad \forall\, \Phi'.\ \Phi' \vdash e_2 \preceq e_2}{e_1 \preceq e_2}$$

Fig. 12. The single-sided symbolic execution judgment.

Single-sided triple. Next is the single-sided triple $\{\Phi\}\ e \preceq e'\ \{\Phi'\}$ (Fig. 12), whose definition (1) quantifies over the implementation-side context, but the specification program, and (2) is "backwards", going from *reduct* e' to *redex* e, as it is meant to transform goals via symbolic execution, preserving refinement with the specification (which remains unchanged) in the process. The context quantified over is a *delimited context* D: an evaluation context possibly nesting further delimited contexts under resets. This allows symbolic execution under enclosing resets; while the meta-context is more appropriate for meta-theory, symbolic execution requires an explicit term representation of resets in contexts. It has a single-sided bind rule (L-BIND) and a soundness rule (L-SOUND) that lifts

it to contextual refinement. The reason for requiring e_2 to be self-related is that $\{\Phi\}\ e \preceq e'\ \{\Phi'\}$ quantifies over some specification program b, which is fixed to e_2 in L-SOUND. It follows from the fundamental theorem when e_2 is a closed term.

A judgment $\{\Phi\}\ e \sqsubseteq_s e'\ \{\Phi'\}$ for executing the specification side can also be defined in an entirely symmetric way, so we present only the left side from here on.

$$\{\Phi\}\ e_1 \preceq e_2\ \{\Phi' ; A\} \triangleq \forall D_1, D_2.\ \big(\forall v_1, v_2.\ A\, v_1\, v_2 \Rrightarrow \Phi' \vdash D_1[v_1] \preceq D_2[v_2]\big) \Rrightarrow \Phi \vdash D_1[e_1] \preceq D_2[e_2]$$

$$A_\Phi \triangleq \lambda v_1\, v_2.\ \Phi \vdash v_1 \preceq v_2 \qquad pin(v, w) \triangleq \lambda v_1\, v_2.\ v_1 = v \wedge v_2 = w$$

REL-BIND

$$\frac{\{\Phi\}\ e_1 \preceq e_2\ \{\Phi_m ; A_m\} \quad \forall v_1, v_2.\ A_m\, v_1\, v_2 \Rrightarrow \{\Phi_m\}\ D_1[v_1] \preceq D_2[v_2]\ \{\Phi' ; A\}}{\{\Phi\}\ D_1[e_1] \preceq D_2[e_2]\ \{\Phi' ; A\}}$$

REL-SOUND

$$\frac{\{emp\}\ e_1 \preceq e_2\ \{\Phi ; A\} \quad \forall v_1\, v_2.\ A\, v_1\, v_2 \Rrightarrow \Phi \vdash v_1 \preceq v_2}{e_1 \preceq e_2}$$

Fig. 13. The double-sided symbolic execution judgment.

Double-sided triple. We may also define a double-sided triple $\{\Phi\}\ e_1 \preceq e_2\ \{\Phi' ; A\}$ (Fig. 13), which quantifies over *both* contexts and is parameterised over an explicit *value relation* A at the results. It has a double-sided bind rule (REL-BIND). There are two useful instances of A. $A_\Phi(v_1, v_2) \triangleq \Phi \vdash v_1 \preceq v_2$ gives a ReLoC-style judgment [Frumin et al. 2021]. The *syntactic pin*, written $pin(v, w)$, fixes both results to given values, making REL-BIND generalise L-BIND. Its soundness theorem REL-SOUND requires the results of e_1 and e_2, which are constrained by A, to be related.

Judgment	Transitive	Congruent	SE left	SE right	Heap
$e_1 \preceq e_2$	✓	any context	rewriting	rewriting	✗
$\Phi \vdash e_1 \preceq e_2$	✗	related eval./meta contexts	✓	✓	✓
$\{\Phi\}\ e_1 \preceq e_2\ \{\Phi'\}$	✓	left delim. contexts	✓	✗	✓
$\{\Phi\}\ e_1 \preceq e_2\ \{\Phi' ; A\}$	pin	both delim. contexts	✓	✓	✓

Fig. 14. Tradeoffs of the different judgments. "SE" = symbolic execution.

Tradeoffs. Fig. 14 summarises the design space of the different judgments. No single one has every desirable property. Contextual refinement allows rewriting freely, even under binders, but cannot talk about the intermediate heap states required for symbolic execution. The other judgments are congruences only at evaluation positions, so they cannot be used to conclude, e.g., $\lambda x.\, e_1 \preceq \lambda x.\, e_2$ from $e_1 \preceq e_2$. The one-sided judgment recovers transitivity, as steps of one program compose easily, and its single-sided bind is useful for applying lemmas and induction hypotheses. The double-sided judgment can symbolically execute on both sides of a refinement, which is useful for refining two heap-manipulating programs. However, double-sided bind is more constraining, and a value relation may need to be provided.

$$\textsc{l-refl}\ \frac{}{\{\Phi\}\ e \preceq e\ \{\Phi\}} \qquad \textsc{l-trans}\ \frac{\{\Phi_1\}\ e_1 \preceq e_2\ \{\Phi_2\} \quad \{\Phi_2\}\ e_2 \preceq e_3\ \{\Phi_3\}}{\{\Phi_1\}\ e_1 \preceq e_3\ \{\Phi_3\}}$$

$$\textsc{l-conseq}\ \frac{\Phi_1 \vdash \Phi_1' \qquad \{\Phi_1'\}\ e \preceq e'\ \{\Phi_2'\} \qquad \Phi_2' \vdash \Phi_2}{\{\Phi_1\}\ e \preceq e'\ \{\Phi_2\}}$$

$$\textsc{l-app-fun}\ \frac{\{\Phi\}\ e_1 \preceq e_1'\ \{\Phi'\}}{\{\Phi\}\ e_1\ e_2 \preceq e_1'\ e_2\ \{\Phi'\}} \qquad \textsc{l-app-arg}\ \frac{\{\Phi_1\}\ e \preceq v\ \{\Phi_2\} \quad \{\Phi_2\}\ f\ v \preceq e'\ \{\Phi_3\}}{\{\Phi_1\}\ f\ e \preceq e'\ \{\Phi_3\}}$$

$$\textsc{l-beta}\ \frac{\mathit{Hered}\ \Phi}{\{\Phi\}\ (\lambda x.\, e)\ v \preceq e[v/x]\ \{\Phi\}}$$

$$\textsc{l-bind}\ \frac{\mathit{Hered}\ \Phi_m \quad \{\Phi\}\ e \preceq v\ \{\Phi_m\} \qquad \{\Phi_m\}\ D[v] \preceq e'\ \{\Phi'\}}{\{\Phi\}\ D[e] \preceq e'\ \{\Phi'\}}$$

$$\textsc{l-let-assoc}\ \frac{\{\Phi\}\ \mathit{let}\ x = a\ \mathit{in}\ (\mathit{let}\ y = b\ \mathit{in}\ c) \preceq e'\ \{\Phi'\}}{\{\Phi\}\ \mathit{let}\ y = (\mathit{let}\ x = a\ \mathit{in}\ b)\ \mathit{in}\ c \preceq e'\ \{\Phi'\}}$$

$$\textsc{l-store}\ \frac{\mathit{Hered}\ \Psi}{\{\ell \mapsto v \star \Psi\}\ \ell := w \preceq ()\ \{\ell \mapsto w \star \Psi\}} \qquad \textsc{l-load}\ \frac{\mathit{Hered}\ \Psi}{\{\ell \mapsto v \star \Psi\}\ !\ell \preceq v\ \{\ell \mapsto v \star \Psi\}}$$

$$\textsc{l-alloc}\ \frac{\mathit{Hered}\ \Psi \qquad \forall\, \ell.\ \{\ell \mapsto v \star \Psi\}\ D[\ell] \preceq e'\ \{\Phi'\}}{\{\Psi\}\ D[\mathit{ref}\ v] \preceq e'\ \{\Phi'\}} \qquad \textsc{l-reset}\ \frac{\{\Phi\}\ e \preceq e'\ \{\Phi'\}}{\{\Phi\}\ \langle e\rangle \preceq \langle e'\rangle\ \{\Phi'\}}$$

$$\textsc{l-reset-shift}\ \frac{\mathit{Hered}\ \Phi}{\{\Phi\}\ \langle K[\mathit{shift}\ k.\, e]\rangle \preceq \langle e[\lambda x.\, \langle K[x]\rangle/k]\rangle\ \{\Phi\}}$$

Fig. 15. Select single-sided symbolic execution rules.

Symbolic execution rules. Next, we cover both single- (Fig. 12) and double-sided (Fig. 13) symbolic execution rules. Symbolic execution works by using congruence rules (L-APP-FUN, L-APP-ARG, and L-BIND) to locate a redex, using one of the reduction rules (L-BETA, L-STORE, L-LOAD, L-ALLOC) to reduce it, then using transitivity (L-TRANS). L-ALLOC has a subtlety: it produces a new location ℓ that cannot make it out into Φ' (nor appear in e') due to the universal quantification. This location (and points-to resource $\ell \mapsto v$) is thus "private" to the premise and must be discarded. There are also structural rules (L-LET-ASSOC, L-REFL, L-CONSEQ) which may be applied manually. The control rules internalise delimited-control reasoning: L-RESET steps under a delimiter, and L-RESET-SHIFT is the triple-level version of SHIFT-RESET, leaving the heap untouched. When the body discards k, the reduct simplifies to $\langle e\rangle$, and the delimiter is dropped by a separate step once e reaches a value or is shift-free. Some rules have hereditary side conditions on assertions, which are required if reduction steps (and world aging) take place. For simplicity, there is no general frame rule, as $\Phi \vdash e_1 \preceq e_2$ evaluates programs against the whole heap (Fig. 12). The rules for heap manipulation thus quantify explicitly over a frame Ψ. This can be remedied by *baking-in* the frame rule [Birkedal et al. 2006]; we take the present approach for simplicity.

Next is the set of double-sided rules (Fig. 16). REL-REFL-PIN is the generalisation of L-REFL which uses the syntactic pin predicate. REL-LEFT lifts many of the single-sided reduction rules, threading the value relation A unchanged. Some rules cannot be lifted, e.g., REL-ALLOC-LEFT, whose fresh location must be universally quantified over a still-double-sided continuation; there is a right one

$$\textsc{rel-refl-pin}\quad \{\Phi\}\ v \preceq w\ \{\Phi\,;\ pin(v,w)\}$$

$$\textsc{rel-left}\quad \frac{\{\Phi\}\ e_1 \preceq r_1\ \{\Phi_m\} \qquad \{\Phi_m\}\ r_1 \preceq e_2\ \{\Phi'\,;\ A\}}{\{\Phi\}\ e_1 \preceq e_2\ \{\Phi'\,;\ A\}}$$

$$\textsc{rel-alloc-left}\quad \frac{Hered\ \Psi \qquad \forall\, \ell.\ \{\ell \mapsto v \star \Psi\}\ D[\ell] \preceq e_2\ \{\Phi\,;\ A\}}{\{\Psi\}\ D[ref\ v] \preceq e_2\ \{\Phi\,;\ A\}}$$

$$\textsc{rel-alloc-tie}\quad \frac{\forall\, W.\ (v_1,v_2) \in \mathcal{V}(W) \qquad Hered\ \Psi \qquad \forall\, \ell_1,\ell_2.\ \{\ell_1 \mapsto v_1 \star \ell_2 \mapsto_s v_2 \star \Psi\}\ D_1[\ell_1] \preceq D_2[\ell_2]\ \{\Phi'\,;\ A\}}{\{\Psi\}\ D_1[ref\ v_1] \preceq D_2[ref\ v_2]\ \{\Phi'\,;\ A\}}$$

$$\textsc{rel-tie}\quad \frac{\forall\, W.\ (v_1,v_2) \in \mathcal{V}(W) \qquad Hered\ \Psi}{\{\ell_1 \mapsto v_1 \star \ell_2 \mapsto_s v_2 \star \Psi\}\ \ell_1 \preceq \ell_2\ \{\ell_1 \mapsto v_1 \star \ell_2 \mapsto_s v_2 \star \Psi\,;\ A_{\ell_1 \mapsto v_1 \star \ell_2 \mapsto_s v_2 \star \Psi}\}}$$

Fig. 16. Double-sided symbolic-execution rules.

as well. Finally, REL-ALLOC-TIE and REL-TIE are rules inapplicable to the single-sided triple, as they concern both heaps, allowing locations across both to be related. REL-ALLOC-TIE allocates a pair of tied locations with related values. REL-TIE allows locations to be tied after they are allocated; the A predicate here says that the locations are related by $\cdot \vdash \cdot \preceq \cdot$ at the post-heap. The $\mathcal{V}$-relatedness of the values and the fact that the precondition carries unchanged into the postcondition is similar to a ReLoC-style invariant.

4.4 Proof Engineering

The judgments of Section 4.3 underlie a verification tool built in Lean for symbolic execution-based refinement proofs. We give an overview of their key ingredients here.

Well-formed syntax. Because we work with a higher-order language, terms e are deeply-embedded, thrice-over.

(1) At the bottom, we use a type of terms with intrinsically-scoped de Bruijn indices, the most pragmatic choice for metatheory. A Lean port of Autosubst 2 [Stark et al. 2019] generates the supporting σ-calculus theory.
(2) On top of that is a locally-nameless layer, intended for proofs. It associates names with binders and uses Lean (d)elaborator machinery to hide the fact that indices are used. The result is that users see de Bruijn terms as named, with names *as they wrote them*, kept fresh automatically.
(3) At the very top is another intrinsically-scoped layer which preserves locally-closed invariants using a subset type. This way, much of the bureaucracy of proving that terms remain closed and well-formed under manipulation of refinements is hidden from the user.

Location-freedom of source programs (Section 4.1) is enforced at layers 2 (with a predicate) and 3 (automatically using a typeclass). There is also automation for discharging wellformedness side conditions at every layer, so this bookkeeping (opening, substitution, closedness, location-freedom, ...) never appears in statements or proofs.

Tactics. For working with contextual refinements, where proofs go largely by rewriting, we use mathlib's generalised rewriting [The mathlib Community 2020]. There are separate tactics

for all the different flavours of symbolic execution: pure reduction (terminating), beta reduction, shift/reset reduction, and heap execution, on the left and right. They are syntax-directed and work uniformly across all the different judgments, dispatching on the shape of the goal, and stopping on unrecognised forms to stay predictable. There is modest heuristic automation which works by composing these tactics. The tactics play well with standard Lean ones, like `apply`, which can be used when lemmas need to be applied manually.

5 Case Studies

In this section, we present an extended case study on a key use case of delimited continuations: *control inversion*, turning a push-based traversal into a pull-based iterator. We first build and verify a persistent iterator (Section 5.1), then an ephemeral one that uses a heap cell (Section 5.2).

5.1 A Persistent Iterator

We begin by inverting a push-based traversal of a binary tree into a persistent, pull-based iterator. Although the core language is untyped, the data involved is best understood through the following OCaml type declarations:

$$\begin{aligned} &\mathit{type}\ \alpha\ \mathit{tree} = \mathit{Leaf}\ \mathit{of}\ \alpha \mid \mathit{Node}\ \mathit{of}\ \alpha\ \mathit{tree} * \alpha\ \mathit{tree} \\ &\mathit{type}\ \alpha\ \mathit{iterator}_p = \mathit{Done} \mid \mathit{Next}\ \mathit{of}\ \alpha * (\mathit{unit} \to \alpha\ \mathit{iterator}_p) \end{aligned}$$

The constructors and pattern matching on them are among the standard features of the core language (Section 3). The programs are shown in Fig. 17.

$$\begin{aligned} &\mathit{let\ rec\ iter}\ f\ t = \\ &\quad \mathit{match}\ t\ \mathit{with} \\ &\quad \mid \mathit{Leaf}\ x \to f\ x \\ &\quad \mid \mathit{Node}\ (t_1, t_2) \to \mathit{let}\ _ = \mathit{iter}\ f\ t_1\ \mathit{in}\ \mathit{iter}\ f\ t_2 \\ &\mathit{let}\ \mathit{yield}_p\ x = \mathit{shift}\ k.\ \mathit{Next}\ (x, k) \\ &\mathit{let}\ \mathit{invert}_p\ t = \langle \mathit{let}\ _ = \mathit{iter}\ \mathit{yield}_p\ t\ \mathit{in}\ \mathit{Done} \rangle \end{aligned}$$

$$\begin{aligned} &\mathit{let\ rec\ tlist}\ t = \\ &\quad \mathit{match}\ t\ \mathit{with} \\ &\quad \mid \mathit{Leaf}\ x \to [x] \\ &\quad \mid \mathit{Node}\ (t_1, t_2) \to \mathit{tlist}\ t_1\ @\ \mathit{tlist}\ t_2 \\ &\mathit{let\ rec\ ilist}_p\ \mathit{it} = \\ &\quad \mathit{match}\ \mathit{it}\ \mathit{with} \\ &\quad \mid \mathit{Done} \to [] \\ &\quad \mid \mathit{Next}\ (x, k) \to x :: \mathit{ilist}_p\ (k\ ()) \end{aligned}$$

Fig. 17. The push-based traversal *iter* and the persistent iterator invert_p

The higher-order function *iter* is the push-based side: it walks the tree t and hands each element, in left-to-right order, to a callback f. The callback we install is yield_p: applied to an element x, it evaluates $\mathit{shift}\ k.\ \mathit{Next}\ (x, k)$, capturing the continuation k and returning $\mathit{Next}\ (x, k)$. k resumes the iteration when applied to $()$. Because $\mathit{invert}_p\ t$ runs $\mathit{iter}\ \mathit{yield}_p\ t$ under a *reset*, the first shift produces the leftmost element and suspends the rest of the traversal; when the traversal is exhausted, the delimited body returns *Done*. The iterator is *persistent*: the captured continuation k is an ordinary function value; it can be applied any number of times, and each application resumes the same traversal.

To state that the iterator enumerates exactly the elements of the tree, we collect the iterator into a list with ilist_p and compare against the result of the pure *tlist*:

$$\forall\, t.\ \mathit{ilist}_p\ (\mathit{invert}_p\ t) \preceq \mathit{tlist}\ t$$

We prove this by generalising over the computation *acc* that runs once the traversal is done, standing for "the rest of the iterator":

$$\forall\, t\ acc.\ ilist_p\ \langle let\ _ = iter\ yield_p\ t\ in\ acc\rangle \preceq tlist\ t\ @\ ilist_p\ \langle acc\rangle$$

The original statement is a corollary with $acc = Done$, since $ilist_p\ \langle Done\rangle$ reduces to [], and [] is the identity of @.

We prove the generalised statement by structural induction on t, with *acc* generalised in the induction hypothesis. In the base case $t = Leaf\ x$, the traversal calls $yield_p$ exactly once, and the goal is discharged by reduction:

$$\begin{array}{ll}
& ilist_p\ \langle let\ _ = iter\ yield_p\ (Leaf\ x)\ in\ acc\rangle & \text{reduce} \\
\preceq & ilist_p\ \langle let\ _ = shift\ k.\ Next\ (x, k)\ in\ acc\rangle & \text{shift-reset, shift-free} \\
\preceq & ilist_p\ (Next\ (x, \lambda_.\ \langle acc\rangle)) & \text{reduce} \\
\preceq & x :: ilist_p\ \langle acc\rangle & \text{fold } tlist \\
\preceq & tlist\ (Leaf\ x)\ @\ ilist_p\ \langle acc\rangle \quad \square &
\end{array}$$

In the inductive case $t = Node\ (t_1, t_2)$, the induction hypothesis quantifies over *acc* and holds for both t_1 and t_2. We first reassociate the two nested traversals so that the hypothesis for t_1 applies against the accumulator $let\ _ = iter\ yield_p\ t_2\ in\ acc$, then apply the hypothesis for t_2 against *acc* itself:

$$\begin{array}{lll}
& ilist_p\ \langle let\ _ = iter\ yield_p\ (Node\ (t_1, t_2))\ in\ acc\rangle & \text{reduce} \\
\preceq & ilist_p\ \langle let\ _ = iter\ yield_p\ t_1\ in\ let\ _ = iter\ yield_p\ t_2\ in\ acc\rangle & \text{IH } (t_1) \\
\preceq & tlist\ t_1\ @\ ilist_p\ \langle let\ _ = iter\ yield_p\ t_2\ in\ acc\rangle & \text{IH } (t_2) \\
\preceq & tlist\ t_1\ @\ tlist\ t_2\ @\ ilist_p\ \langle acc\rangle & \text{fold } tlist \\
\preceq & tlist\ (Node\ (t_1, t_2))\ @\ ilist_p\ \langle acc\rangle \quad \square &
\end{array}$$

5.2 An Ephemeral Iterator

The persistent iterator of Section 5.1 materialises every suspended continuation as a value. Using a single heap cell, we can instead build an *ephemeral* iterator: a function that, on each call, returns *Some x* for the next element *x*, or *None* when the traversal is exhausted:

$$type\ \alpha\ iterator_e = unit \to \alpha\ option$$

The programs are shown in Fig. 18.

```
let invert_e t =
  let r = ref (λ_. None) in
  let yield_e x = shift k. let _ = r := k in Some x in
  let _ = r := λ_. ⟨let _ = iter yield_e t in None⟩ in
  λ_. !r ()
```

```
let ilist_e it =
  let rec ilist'_e opt =
    match opt with
    | None → []
    | Some x → x :: ilist'_e (it ())
  in
  ilist'_e (it ())
```

Fig. 18. The ephemeral iterator $invert_e$, using a single heap cell

The reference *r* is initialised with a function that returns *None*, then overwritten with the initial state of the traversal: a thunk that runs $iter\ yield_e\ t$ under a *reset* and returns *None* once the traversal is complete. The iterator itself is the function $\lambda_.\ !r\ ()$, which invokes the suspended traversal that is currently stored in *r*; performing the shift inside $yield_e$ saves the rest of the traversal into *r* and

returns *Some* x. Unlike its persistent counterpart, this iterator is stateful: each call advances the state of the traversal. The specification is as before, consuming the iterator with $ilist_e$:

$$\forall\, t.\ ilist_e\ (invert_e\ t) \preceq tlist\ t$$

As with $toss_c$ (Section 2.2), $\preceq$ cannot directly handle the cell r, so we again go through the program logic of Section 4.3. Applying L-SOUND reduces the goal to deriving:

$$\{emp\}\ ilist_e\ (invert_e\ t) \preceq tlist\ t\ \{\Phi'\}$$

for some postcondition Φ'.

We symbolically execute $invert_e\ t$. The allocation of r (L-ALLOC) introduces a fresh location holding $\lambda_.\ None$, and then the following assignment (L-STORE) immediately overwrites it with the initial traversal thunk; $invert_e\ t$ then returns the iterator value $\lambda_.\ !r\ ()$. By further beta reductions and a dereference (L-LOAD), we have:

$$\{r \mapsto \lambda_.\ \langle let\ _ = iter\ yield_e\ t\ in\ None\rangle\}\ ilist'_e\ \langle let\ _ = iter\ yield_e\ t\ in\ None\rangle \preceq tlist\ t\ \{\Phi'\}$$

A direct induction on this goal gets stuck, just as for $toss_c$, so we generalise over the initial content k of the cell r and over the computation acc standing for "the rest of the iterator":

$$\forall\, t\ k\ acc.\ \{r \mapsto k\}\ ilist'_e\ \langle let\ _ = iter\ yield_e\ t\ in\ acc\rangle \preceq tlist\ t\ @\ ilist'_e\ \langle acc\rangle\ \{r \mapsto \lambda_.\ \langle acc\rangle\}$$

The goal above follows as a corollary, with $acc = None$ and k the initial traversal thunk: $ilist'_e\ \langle None\rangle$ reduces to $[]$, so the right-hand side collapses to $tlist\ t\ @\ [] = tlist\ t$, matching the goal.

We prove the generalised statement by structural induction on t, with k and acc generalised. In the base case $t = Leaf\ x$, the traversal calls $yield_e$ exactly once, capturing the continuation $\lambda_.\ \langle acc\rangle$ and storing it into r:

$$\begin{array}{ll}
& \{r \mapsto k\} \\
\preceq & ilist'_e\ \langle let\ _ = iter\ yield_e\ (Leaf\ x)\ in\ acc\rangle \quad \text{reduce, shift-free} \\
& \{r \mapsto k\} \\
\preceq & ilist'_e\ (let\ _ = r := \lambda_.\ \langle acc\rangle\ in\ Some\ x) \quad \text{store, reduce} \\
& \{r \mapsto \lambda_.\ \langle acc\rangle\} \\
\preceq & x :: ilist'_e\ (!r\ ()) \quad \text{load, reduce} \\
& \{r \mapsto \lambda_.\ \langle acc\rangle\} \\
\preceq & x :: ilist'_e\ \langle acc\rangle \quad \text{fold } tlist \\
& \{r \mapsto \lambda_.\ \langle acc\rangle\} \\
\preceq & tlist\ (Leaf\ x)\ @\ ilist'_e\ \langle acc\rangle \\
& \{r \mapsto \lambda_.\ \langle acc\rangle\} \quad \square
\end{array}$$

In the inductive case $t = Node\ (t_1, t_2)$, the induction hypothesis quantifies over k and acc, and holds for both t_1 and t_2:

$$\begin{array}{ll}
& \{r \mapsto k\} \\
\preceq & ilist'_e\ \langle let\ _ = iter\ yield_e\ (Node\ (t_1, t_2))\ in\ acc\rangle \quad \text{reduce} \\
& \{r \mapsto k\} \\
\preceq & ilist'_e\ \langle let\ _ = iter\ yield_e\ t_1\ in\ let\ _ = iter\ yield_e\ t_2\ in\ acc\rangle \quad \text{IH } (t_1) \\
& \{r \mapsto \lambda_.\ \langle let\ _ = iter\ yield_e\ t_2\ in\ acc\rangle\} \\
\preceq & tlist\ t_1\ @\ ilist'_e\ \langle let\ _ = iter\ yield_e\ t_2\ in\ acc\rangle \quad \text{IH } (t_2) \\
& \{r \mapsto \lambda_.\ \langle acc\rangle\} \\
\preceq & tlist\ t_1\ @\ tlist\ t_2\ @\ ilist'_e\ \langle acc\rangle \quad \text{fold } tlist \\
& \{r \mapsto \lambda_.\ \langle acc\rangle\} \\
\preceq & tlist\ (Node\ (t_1, t_2))\ @\ ilist'_e\ \langle acc\rangle \\
& \{r \mapsto \lambda_.\ \langle acc\rangle\} \quad \square
\end{array}$$

The two applications of the induction hypothesis differ in how *acc* is instantiated: for t_1, with the rest of the traversal $\mathit{let}\ _ = \mathit{iter}\ \mathit{yield}_e\ t_2\ \mathit{in}\ \mathit{acc}$, so that the postcondition leaves r holding precisely the thunk for the remaining traversal; for t_2, with *acc* itself. In both, k is instantiated with the current content of r, which may be arbitrary at the point of application: the traversal always overwrites r before the iterator reads it.

6 Evaluation

We evaluate the verification tool described in Section 4.4 on a number of case studies, which are mechanised end-to-end in Lean, without axioms or sorrys. Each establishes a contextual refinement as the final result. Fig. 19 gives an overview of the suite. Several examples are adapted from the literature on delimited continuations; these are necessarily higher-order. The rest stress the interaction between mutable state, higher-order functions, and multishot continuations.

Program	**Features**	**Def.**	**Proof**
Foldr	H, R	13	29
FoldrClosure [Foo et al. 2024]	H, R, S	29	82
Append [Danvy and Filinski 1989]	H, R, C	27	53
Reverse [Hirota and Asai 2011]	H, R, C	25	62
Times [Asai and Kiselyov 2011]	R, C	29	64
Toss [Song et al. 2024]	R, C, M, S	66	220
Regex	R, C, M	97	414
Invert	H, R, C	19	17
InvertTree	H, R, C	41	84
InvertTreeEph	H, R, C, S	57	170
Landin [Landin 1964]	H, R, S	23	57
Equiv [Sharma et al. 2013]	H, R, S	28	71
Total		454	1323

Fig. 19. Case studies. Features: **H** higher-order functions, **R** recursion (object-level fixpoint or meta-level list/nat recursion), **C** delimited control (shift/reset), **M** multishot continuations, **S** mutable state. Def. and Proof count non-blank, non-comment lines of program definitions and of theorem statements plus proofs, respectively.

Case studies. Foldr is a simple stress test of a recursive higher-order function; FoldrClosure extends it with a stateful closure, showing that the system can be used for complex stateful higher-order functions. This line of examples culminates in Landin's knot, which has been verified before [Birkedal and Bizjak 2023], though not in such a simple manner: we use symbolic execution rather than an invariant.

Reverse and Times were verified by Hirota and Asai [2011] in Rocq using the generalised monad [Kiselyov 2007; Sozeau 2008]. The statements are much more complex compared to our untyped treatment, due to the shallow embedding and, consequently, the need to supply polymorphic answer types. To our knowledge, the rest have not been foundationally verified before. The amount of effort required was generally low given the complexity of some of these programs, with a proof-to-definition ratio of 2.91.

Proof techniques. A typical technique which works for pure, recursive shift/reset programs is to generalise the induction hypothesis over a continuation under the delimiter, then use E-LET-ASSOC to

reassociate before applying the induction hypotheses. While there is a similar rule in the program logic for stateful programs (L-LET-ASSOC), it is not directly available in the mechanisation for technical reasons. The rule is thus encoded instead via a reification of the evaluation context as a stack of bind frames, which can be seen as a form of defunctionalization. This is necessary because the context is dynamic and changes across the recursive calls. The same technique is used in the ephemeral iterator case study.

7 Related Work

To our knowledge, this is the first mechanised framework for reasoning about shift and reset directly, i.e., in a protocol-free manner. It unifies two lines of work: one on semantics, models, and reasoning for shift and reset, and another on the construction of higher-order and relational program logics.

Semantics and models for shift and reset. The semantics of shift and reset was originally given through the CPS transformation [Danvy and Filinski 1990]. A small-step operational semantics was first given in Biernacka et al. [2005], which the authors augmented with a meta-context. This directly inspired the approach in our paper, which uses this semantics as the basis for an operational logical relation. A logical relation for a typed lambda calculus with shift/reset but without state has been proposed before [Asai 2005], but not mechanised.

Reasoning about shift and reset. A sound and complete axiomatization for shift and reset was given by Kameyama and Hasegawa [2003], and later extended to the CPS hierarchy [Kameyama 2004]. The laws proposed directly inspired the ones for the shift/reset fragment of the contextual refinement logic in Section 4.1. Two other families of approaches to reasoning about shift and reset are bisimulations [Biernacki et al. 2019]. and type-and-effect systems [Asai and Kameyama 2007; Danvy and Filinski 1989; Kawamata et al. 2024; Sekiyama and Unno 2023]. There is a known correspondence between bisimulations and logical relations [Hermida et al. 2022; Hur et al. 2012], and it would be interesting future work to see how the insights of our logical relation can be transferred to the bisimulation setting. A key feature of type systems for shift/reset programs is answer types. Answer types are an abstraction of the delimited context, similar to the protocols in program logics. When augmented with refinements [Kawamata et al. 2024], they can be used to prove functional correctness, but only of pure programs. A typed approach would necessitate dealing with answer types, hence this work uses an untyped presentation for simplicity. We return to protocols shortly.

Higher-order, relational program logics. Modern higher-order program logics are built atop sophisticated step-indexed logical relations, and so predominantly use Iris [Jung et al. 2018]. This line of relational logics descends from the logical relations of Turon et al. [2013b] and CaReSL [Turon et al. 2013a], which first unified refinement and Hoare-style reasoning. The closest framework for relational reasoning on higher-order programs is thus the Iris-based ReLoC [Frumin et al. 2018], latterly ReLoC Reloaded [Frumin et al. 2021]. Our main judgment is very similar, though it is a proposition of the (Lean) metalogic, as opposed to an Iris proposition. Most of the differences in both frameworks follow from this design choice. Unlike in Iris, our main judgment cannot talk about resources intrinsically, and so requires explicit heap pre- and postconditions. We also define the judgment in terms of a (CPS formulation of) refinement directly, compared to using an invariant, the wand-and-weakest-precondition formulation, and a ghost program as the specification. For these reasons, our treatment is more elementary and lightweight (allowing us to use the simpler Indirection Theory [Hobor et al. 2010] as a foundation), at the cost of expressiveness relating to the use of Iris' features, like invariants, for, e.g., concurrency. Another key difference is that we

propose a second judgment with one-sided symbolic execution, but which recovers transitivity as a tradeoff. Dreyer et al. [2012] study in depth how higher-order state and control effects impact local step-indexed relational reasoning, in a typed setting with undelimited continuations. Balik et al. [2025] also study a step-indexed logical relation for an untyped lambda calculus, but use undelimited continuations. Our work can be seen as an extension of theirs to delimited continuations, with the addition of a program logic for reasoning about the heap.

Program logics for delimited continuations. There is a known correspondence between shift/reset and effect handlers [Forster et al. 2019; Piróg et al. 2019]. For this reason, a program logic for effect handlers [de Vilhena and Pottier 2021] can effectively be used to verify shift/reset programs. de Vilhena [2022] showed how to handle the undelimited operators callcc and throw in this framework, via an encoding using a toplevel handler, as well as shift and reset, via an "answer postcondition", a specification for the enclosing reset. The key feature of this approach is the *effectful weakest precondition*, which augments the *wp* of Iris with a *protocol*: a pre- and postcondition for an effect. Together with a *handler invariant* [Soares and Pereira 2023], it is possible to verify both the effect use site and handler independently of each other, against the protocol. This program logic has since been generalised to a relational one [de Vilhena et al. 2026], and the protocol to a *relational theory*. The key difference of our work is the absence of the protocol entirely: we only resolve a shift once the extent of its reset is known. Such a specification is modular in a different sense: even though it contains less information, it does not require the behaviour of the reset to be abstracted upfront; we instead abstract it on demand based on the way a shift is used. We argue that this per-method strategy enhances both modularity and automation.

8 Conclusion

In this paper, we presented a program logic for verifying higher-order, imperative programs with the delimited control operators shift and reset. It is built using a novel biorthogonal logical relation for shift and reset, a useful semantic ingredient in itself. It also enables simple proofs based on symbolic execution, without auxiliary specifications like protocols. We have demonstrated the soundness and practical utility of the approach by mechanising the logic in Lean, implementing a verifier atop the mechanisation, and verifying a suite of case studies. The results demonstrate that the approach is readily usable for proving functional correctness of this class of programs.

References

Andrew W. Appel and David A. McAllester. 2001. An indexed model of recursive types for foundational proof-carrying code. *ACM Trans. Program. Lang. Syst.* 23, 5 (2001), 657–683. https://doi.org/10.1145/504709.504712

Kenichi Asai. 2005. Logical relations for call-by-value delimited continuations. In *Revised Selected Papers from the Sixth Symposium on Trends in Functional Programming, TFP 2005, Tallinn, Estonia, 23-24 September 2005 (Trends in Functional Programming, Vol. 6)*, Marko C. J. D. van Eekelen (Ed.). Intellect, 63–78.

Kenichi Asai and Yukiyoshi Kameyama. 2007. Polymorphic Delimited Continuations. In *Programming Languages and Systems, 5th Asian Symposium, APLAS 2007, Singapore, November 29-December 1, 2007, Proceedings (Lecture Notes in Computer Science, Vol. 4807)*, Zhong Shao (Ed.). Springer, 239–254. https://doi.org/10.1007/978-3-540-76637-7_16

Kenichi Asai and Oleg Kiselyov. 2011. Introduction to Programming with Shift and Reset. https://api.semanticscholar.org/CorpusID:62358634

Patrycja Balik, Dariusz Biernacki, and Piotr Polesiuk. 2025. Untyped Logical Relations at Work: Control Operators, Contextual Equivalence and Full Abstraction. In *Proceedings of the Workshop Dedicated to Olivier Danvy on the Occasion of His 64th Birthday* (Singapore, Singapore) *(OLIVIERFEST '25)*. Association for Computing Machinery, New York, NY, USA, 128–141. https://doi.org/10.1145/3759427.3760374

Małgorzata Biernacka, Dariusz Biernacki, and Olivier Danvy. 2005. An Operational Foundation for Delimited Continuations in the CPS Hierarchy. *Logical Methods in Computer Science* 1, 2:5 (Nov. 2005), 1–39.

Dariusz Biernacki, Sergueï Lenglet, and Piotr Polesiuk. 2019. Bisimulations for Delimited-Control Operators. *Log. Methods Comput. Sci.* 15, 2 (2019). https://doi.org/10.23638/LMCS-15(2:18)2019

Lars Birkedal and Aleš Bizjak. 2023. Lecture Notes on Iris: Higher-order Concurrent Separation Logic. *Lectures notes, September* (2023). https://iris-project.org/tutorial-material.html

Lars Birkedal, Noah Torp-Smith, and Hongseok Yang. 2006. Semantics of Separation-Logic Typing and Higher-order Frame Rules for Algol-like Languages. *Log. Methods Comput. Sci.* 2, 5 (2006). https://doi.org/10.2168/LMCS-2(5:1)2006

Olivier Danvy and Andrzej Filinski. 1989. *A Functional Abstraction of Typed Contexts.* DIKU Rapport 89/12. DIKU, Computer Science Department, University of Copenhagen, Copenhagen, Denmark.

Olivier Danvy and Andrzej Filinski. 1990. Abstracting Control. In *Proceedings of the 1990 ACM Conference on LISP and Functional Programming, LFP 1990, Nice, France, 27-29 June 1990*, Gilles Kahn (Ed.). ACM, 151–160. https://doi.org/10.1145/91556.91622

Paulo de Vilhena. 2022. *Proof of Programs with Effect Handlers. (Preuve de Programmes avec Effect Handlers).* Ph. D. Dissertation. Paris Cité University, France. https://tel.archives-ouvertes.fr/tel-03891381

Paulo Emílio de Vilhena and François Pottier. 2021. A separation logic for effect handlers. *Proc. ACM Program. Lang.* 5, POPL (2021), 1–28. https://doi.org/10.1145/3434314

Paulo Emílio de Vilhena, Simcha van Collem, Ines Wright, and Robbert Krebbers. 2026. A Relational Separation Logic for Effect Handlers. *Proc. ACM Program. Lang.* 10, POPL (2026), 981–1009. https://doi.org/10.1145/3776676

Germán Andrés Delbianco and Aleksandar Nanevski. 2013. Hoare-style reasoning with (algebraic) continuations. In *ACM SIGPLAN International Conference on Functional Programming, ICFP'13, Boston, MA, USA - September 25 - 27, 2013*, Greg Morrisett and Tarmo Uustalu (Eds.). ACM, 363–376. https://doi.org/10.1145/2500365.2500593

Derek Dreyer, Amal Ahmed, and Lars Birkedal. 2011. Logical Step-Indexed Logical Relations. *Log. Methods Comput. Sci.* 7, 2 (2011). https://doi.org/10.2168/LMCS-7(2:16)2011

Derek Dreyer, Georg Neis, and Lars Birkedal. 2012. The impact of higher-order state and control effects on local relational reasoning. *J. Funct. Program.* 22, 4-5 (2012), 477–528. https://doi.org/10.1017/S095679681200024X

Matthew Flatt, Gang Yu, Robert Bruce Findler, and Matthias Felleisen. 2007. Adding delimited and composable control to a production programming environment. In *Proceedings of the 12th ACM SIGPLAN International Conference on Functional Programming, ICFP 2007, Freiburg, Germany, October 1-3, 2007*, Ralf Hinze and Norman Ramsey (Eds.). ACM, 165–176. https://doi.org/10.1145/1291151.1291178

Darius Foo, Yahui Song, and Wei-Ngan Chin. 2024. Staged Specification Logic for Verifying Higher-Order Imperative Programs. In *FM 2024: Formal Methods - 17th International Symposium on Formal Methods, Milan, Italy, Sept 9-13, 2024. Proceedings (Lecture Notes in Computer Science)*. Springer.

Yannick Forster, Ohad Kammar, Sam Lindley, and Matija Pretnar. 2019. On the expressive power of user-defined effects: Effect handlers, monadic reflection, delimited control. *J. Funct. Program.* 29 (2019), e15. https://doi.org/10.1017/S0956796819000121

Dan Frumin, Robbert Krebbers, and Lars Birkedal. 2018. ReLoC: A Mechanised Relational Logic for Fine-Grained Concurrency. In *Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science, LICS 2018, Oxford, UK, July 09-12, 2018*, Anuj Dawar and Erich Grädel (Eds.). ACM, 442–451. https://doi.org/10.1145/3209108.3209174

Dan Frumin, Robbert Krebbers, and Lars Birkedal. 2021. ReLoC Reloaded: A Mechanized Relational Logic for Fine-Grained Concurrency and Logical Atomicity. *Log. Methods Comput. Sci.* 17, 3 (2021). https://doi.org/10.46298/LMCS-17(3:9)2021

Claudio Hermida, Uday S. Reddy, Edmund Robinson, and Alessio Santamaria. 2022. Bisimulation as a logical relation. *Math. Struct. Comput. Sci.* 32, 4 (2022), 442–471. https://doi.org/10.1017/S0960129522000020

Noriko Hirota and Kenichi Asai. 2011. Correctness of functions with shift and reset. In *Proceedings of the 2011 ACM SIGPLAN Continuation Workshop*. 23–25. https://www.cs.tsukuba.ac.jp/~kam/cw2011/book.pdf

Aquinas Hobor, Robert Dockins, and Andrew W. Appel. 2010. A theory of indirection via approximation. In *Proceedings of the 37th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2010, Madrid, Spain, January 17-23, 2010*, Manuel V. Hermenegildo and Jens Palsberg (Eds.). ACM, 171–184. https://doi.org/10.1145/1706299.1706322

Chung-Kil Hur, Derek Dreyer, Georg Neis, and Viktor Vafeiadis. 2012. The marriage of bisimulations and Kripke logical relations. In *Proceedings of the 39th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2012, Philadelphia, Pennsylvania, USA, January 22-28, 2012*, John Field and Michael Hicks (Eds.). ACM, 59–72. https://doi.org/10.1145/2103656.2103666

Ralf Jung, Robbert Krebbers, Jacques-Henri Jourdan, Ales Bizjak, Lars Birkedal, and Derek Dreyer. 2018. Iris from the ground up: A modular foundation for higher-order concurrent separation logic. *J. Funct. Program.* 28 (2018), e20. https://doi.org/10.1017/S0956796818000151

Yukiyoshi Kameyama. 2004. Axioms for Delimited Continuations in the CPS Hierarchy. In *Computer Science Logic, 18th International Workshop, CSL 2004, 13th Annual Conference of the EACSL, Karpacz, Poland, September 20-24, 2004, Proceedings (Lecture Notes in Computer Science, Vol. 3210)*, Jerzy Marcinkowski and Andrzej Tarlecki (Eds.). Springer, 442–457. https://doi.org/10.1007/978-3-540-30124-0_34

Yukiyoshi Kameyama and Masahito Hasegawa. 2003. A sound and complete axiomatization of delimited continuations. In *Proceedings of the Eighth ACM SIGPLAN International Conference on Functional Programming, ICFP 2003, Uppsala, Sweden,*

August 25-29, 2003, Colin Runciman and Olin Shivers (Eds.). ACM, 177–188. https://doi.org/10.1145/944705.944722

Fuga Kawamata, Hiroshi Unno, Taro Sekiyama, and Tachio Terauchi. 2024. Answer Refinement Modification: Refinement Type System for Algebraic Effects and Handlers. *Proc. ACM Program. Lang.* 8, POPL (2024), 115–147. https://doi.org/10.1145/3633280

Oleg Kiselyov. 2007. Genuine shift/reset in Haskell98. https://okmij.org/ftp/continuations/implementations.html.

Oleg Kiselyov. 2010. Delimited Control in OCaml, Abstractly and Concretely: System Description. In *Functional and Logic Programming, 10th International Symposium, FLOPS 2010, Sendai, Japan, April 19-21, 2010. Proceedings (Lecture Notes in Computer Science, Vol. 6009)*, Matthias Blume, Naoki Kobayashi, and Germán Vidal (Eds.). Springer, 304–320. https://doi.org/10.1007/978-3-642-12251-4_22

P. J. Landin. 1964. The Mechanical Evaluation of Expressions. *Comput. J.* 6, 4 (1964), 308–320. https://doi.org/10.1093/COMJNL/6.4.308

Marek Materzok and Dariusz Biernacki. 2011. Subtyping delimited continuations. In *Proceeding of the 16th ACM SIGPLAN international conference on Functional Programming, ICFP 2011, Tokyo, Japan, September 19-21, 2011*, Manuel M. T. Chakravarty, Zhenjiang Hu, and Olivier Danvy (Eds.). ACM, 81–93. https://doi.org/10.1145/2034773.2034786

Maciej Piróg, Piotr Polesiuk, and Filip Sieczkowski. 2019. Typed Equivalence of Effect Handlers and Delimited Control. In *4th International Conference on Formal Structures for Computation and Deduction, FSCD 2019, Dortmund, Germany, June 24-30, 2019 (LIPIcs, Vol. 131)*, Herman Geuvers (Ed.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 30:1–30:16. https://doi.org/10.4230/LIPICS.FSCD.2019.30

Tiark Rompf, Ingo Maier, and Martin Odersky. 2009. Implementing first-class polymorphic delimited continuations by a type-directed selective CPS-transform. In *Proceeding of the 14th ACM SIGPLAN international conference on Functional programming, ICFP 2009, Edinburgh, Scotland, UK, August 31 - September 2, 2009*, Graham Hutton and Andrew P. Tolmach (Eds.). ACM, 317–328. https://doi.org/10.1145/1596550.1596596

Taro Sekiyama and Hiroshi Unno. 2023. Temporal Verification with Answer-Effect Modification: Dependent Temporal Type-and-Effect System with Delimited Continuations. *Proc. ACM Program. Lang.* 7, POPL (2023), 2079–2110. https://doi.org/10.1145/3571264

Rahul Sharma, Saurabh Gupta, Bharath Hariharan, Alex Aiken, and Aditya V. Nori. 2013. Verification as Learning Geometric Concepts. In *Static Analysis - 20th International Symposium, SAS 2013, Seattle, WA, USA, June 20-22, 2013. Proceedings (Lecture Notes in Computer Science, Vol. 7935)*, Francesco Logozzo and Manuel Fähndrich (Eds.). Springer, 388–411. https://doi.org/10.1007/978-3-642-38856-9_21

Tiago Soares and Mário Pereira. 2023. A Framework for the Automated Verification of Algebraic Effects and Handlers (extended version). *CoRR* abs/2302.01265 (2023). https://doi.org/10.48550/ARXIV.2302.01265 arXiv:2302.01265

Yahui Song, Darius Foo, and Wei-Ngan Chin. 2024. Specification and Verification for Unrestricted Algebraic Effects and Handling. *Proc. ACM Program. Lang.* ICFP (2024).

Matthieu Sozeau. 2008. The Proved Program of The Month - Type-safe printf via delimited continuations. https://web.archive.org/web/20080513223632/http://www.lri.fr/perso/~sozeau/repos/coq/misc/shiftreset/GenuineShiftReset.html.

Kathrin Stark, Steven Schäfer, and Jonas Kaiser. 2019. Autosubst 2: reasoning with multi-sorted de Bruijn terms and vector substitutions. In *Proceedings of the 8th ACM SIGPLAN International Conference on Certified Programs and Proofs, CPP 2019, Cascais, Portugal, January 14-15, 2019*, Assia Mahboubi and Magnus O. Myreen (Eds.). ACM, 166–180. https://doi.org/10.1145/3293880.3294101

The mathlib Community. 2020. The Lean Mathematical Library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*. ACM, New Orleans, LA, USA. https://doi.org/10.1145/3372885.3373824

Amin Timany and Lars Birkedal. 2019. Mechanized relational verification of concurrent programs with continuations. *Proc. ACM Program. Lang.* 3, ICFP (2019), 105:1–105:28. https://doi.org/10.1145/3341709

Aaron Turon, Derek Dreyer, and Lars Birkedal. 2013a. Unifying refinement and hoare-style reasoning in a logic for higher-order concurrency. In *ACM SIGPLAN International Conference on Functional Programming, ICFP'13, Boston, MA, USA - September 25 - 27, 2013*, Greg Morrisett and Tarmo Uustalu (Eds.). ACM, 377–390. https://doi.org/10.1145/2500365.2500600

Aaron Joseph Turon, Jacob Thamsborg, Amal Ahmed, Lars Birkedal, and Derek Dreyer. 2013b. Logical relations for fine-grained concurrency. In *The 40th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL '13, Rome, Italy - January 23 - 25, 2013*, Roberto Giacobazzi and Radhia Cousot (Eds.). ACM, 343–356. https://doi.org/10.1145/2429069.2429111